ChatGPT Conversation Export

Video Transcription Facts — Machine Learning Lectures

Source: chatgpt.com/share/6ae600ae-f578-83ec-ae0c-37bc0ff13ed3

USER (Message 1)

https://youtube.com/playlist?list=PLKnIA16_Rmvbr7zKYQuBfsVkjoLcJgxHH&si=WlkpkI6oMi9PU5lZå■¯ like video creators your job is to like make sure you create according to you, you need to transfer the video. And on the basis of that you give me the detalled exp about every video. And make sure the video like you read the video completely. You read the transcript completely. And yeah from 31 lec to next one by one
Firstly give for 31
Language should be simple and easy to understand
So that i read it once and get it
I will read when i dont have internet so i cant searvh in between

CHATGPT (Message 2)

We'll prepare this as a **complete, offline-friendly study guide**, going through the playlist one lecture at a time, starting with Lecture 31.

My goal will be to make the notes detailed enough that you can read them without opening YouTube or searching Google.

How I'll prepare each lecture

1. **Complete lecture coverage:** Follow the lecture from beginning to end, not just its title or a short summary.
2. **Simple language:** Explain every concept in beginner-friendly English.
3. **Detailed explanations:** Explain what something is, why it is needed, how it works, and where it is used.
4. **Code examples:** Include relevant code, with explanations of important lines.
5. **Examples:** Use practical examples to make difficult concepts easy to understand.
6. **Important distinctions:** Explain concepts that students commonly confuse.
7. **Revision section:** Include key points, definitions and questions at the end.

I'll avoid unnecessary analogies and unexplained technical terminology. The notes should read like a proper textbook chapter.

CHATGPT (Message 3)

We'll prepare this as a **complete, offline-friendly study guide**, going through the playlist one lecture at a time, starting with Lecture 31.

My goal will be to make the notes detailed enough that you can read them without opening YouTube or searching Google.

How I'll prepare each lecture

1. **Complete lecture coverage:** Follow the lecture from beginning to end, not just its title or a short summary.
2. **Simple language:** Explain every concept in beginner-friendly English.
3. **Detailed explanations:** Explain what something is, why it is needed, how it works, and where it is used.

4. **Code examples:** Include relevant code, with explanations of important lines.
5. **Examples:** Use practical examples to make difficult concepts easy to understand.
6. **Important distinctions:** Explain concepts that students commonly confuse.
7. **Revision section:** Include key points, definitions and questions at the end.

I'll avoid unnecessary analogies and unexplained technical terminology. The notes should read like a proper textbook chapter.

Lecture 31 — Power Transformer
Box-Cox Transformation and Yeo-Johnson Transformation

[Lecture identified]
17 April 2021 · 21 min 28 sec

I verified that Lecture 31 is about Power Transformation, specifically Box-Cox and Yeo-Johnson transformations.

One important thing before I write the chapter: I can identify the lecture, but I haven't been able to retrieve its complete transcript. I don't want to pretend I've read all 21 minutes when I haven't.

Please send me the transcript of Lecture 31.

On YouTube:
1. Open Lecture 31.
2. Expand its description.
3. Select **Show transcript**.
4. Copy the transcript and paste it here. You can send it in multiple messages if necessary.

I'll turn it into complete, detailed notes, preserving the lecturer's topics and adding explanations wherever needed.

Our rule for all 134 lectures: No skipping concepts, no superficial summaries, and no claiming transcript coverage without actually having the transcript.

We'll proceed sequentially: 31 → 32 → 33 → onwards.

CHATGPT (Message 4)

Lecture 31 — Power Transformer
Box-Cox Transformation and Yeo-Johnson Transformation

[Lecture identified]
17 April 2021 · 21 min 28 sec

I verified that Lecture 31 is about Power Transformation, specifically Box-Cox and Yeo-Johnson transformations.

One important thing before I write the chapter: I can identify the lecture, but I haven't been able to retrieve its complete transcript. I don't want to pretend I've read all 21 minutes when I haven't.

Please send me the transcript of Lecture 31.

On YouTube:
1. Open Lecture 31.
2. Expand its description.
3. Select **Show transcript**.
4. Copy the transcript and paste it here. You can send it in multiple messages if necessary.

I'll turn it into complete, detailed notes, preserving the lecturer's topics and adding explanations wherever needed.

Our rule for all 134 lectures: No skipping concepts, no superficial summaries, and no claiming transcript coverage without actually having the transcript.

We'll proceed sequentially: 31 → 32 → 33 → onwards.

[Attachment: Power Transformer Box - Cox Transform Yeo - Johnson Transform.txt (46971 bytes)]

CHATGPT (Message 5)

Lecture 31: Power Transformation

Machine Learning — Feature Transformation | Complete study notes

[Transcript received and covered]

This chapter is based on the complete transcript you uploaded. I've organized the lecture into a proper learning chapter, explained the concepts in simple English, and expanded the technical points so that you can study without an internet connection.

The lecture covers three important things:

1. Power Transformation
2. Box-Cox Transformation
3. Yeo-Johnson Transformation

It also demonstrates these transformations using a real-world concrete strength dataset and Linear Regression.

---

1. Why do we need mathematical transformations?

Before understanding Power Transformation, we need to understand why mathematical transformations are required in Machine Learning.

Suppose you have a dataset with different input features:

Cement	Water	Age	Concrete strength
250	180	28	35
350	200	90	52
150	160	7	20
400	190	56	60

Our objective is to predict concrete strength using the input features.

This is a **Regression Problem** because the output (concrete strength) is a continuous numerical value.

We can train a Linear Regression model on this dataset.

However, there is one possible problem.

The input features may not follow a Normal Distribution.

For example:
- Some features may have most values concentrated around smaller numbers.
- A few observations may have extremely large values.
- The distribution may be heavily skewed towards the right or left.

These distributions can affect the performance of certain machine learning algorithms.

This is where mathematical transformations become useful.

1.1 What is a mathematical transformation?

A mathematical transformation means changing the values of a feature using a mathematical function.

For example, suppose a feature contains:

$$
X=[1,4,9,16,25]
$$

If we apply a square-root transformation:

$$
X'=\sqrt{X}
$$

The transformed values become:

$$
```
```

X'=[1,2,3,4,5]
\]

Notice that the original values have changed, but the observations are still in the same order.

We have simply represented the data differently.

Why do we do this?

To make the distribution of a feature closer to a Normal Distribution.

This is particularly useful when working with algorithms that benefit from approximately normally distributed features.

Examples include:
- Linear Regression (in certain modelling situations).
- Logistic Regression.
- Some statistical models that make distributional assumptions.

An important distinction: Normality of input features is not a universal requirement for Linear Regression or Logistic Regression. Transformations can help, but they do not automatically guarantee a better model.

---

2. Types of Mathematical Transformations

In the previous lecture, the instructor discussed Function Transformer.

There are two major categories in this part of the syllabus.

1. Function Transformer

It allows us to apply mathematical functions such as:

- Log transformation
- Square-root transformation
- Reciprocal transformation

2. Power Transformer

It provides two important transformations:

- Box-Cox Transformation
- Yeo-Johnson Transformation

The main difference is that Power Transformer can estimate the appropriate power parameter from the data instead of requiring us to manually choose a mathematical function.

---

3. What is Power Transformation?

Power Transformation is a family of mathematical transformations used to make data more closely resemble a Normal Distribution.

It automatically determines an appropriate power parameter, usually represented by the Greek letter lambda:

$$
\lambda
$$

The important question is:

Which power should we apply to our feature?

Should we use:

$$
X^2
$$

Or:

$$
X^3
$$

Or:

$$
\sqrt X
$$

Or:

$$
\log(X)
$$

We usually don't know which transformation will work best.

Power Transformation attempts to find a suitable value of lambda based on the data.

3.1 Understanding Lambda (λ)

Lambda represents the power parameter used in the transformation.

For example:

Lambda	Approximate transformation
-2	Reciprocal square

-1	Reciprocal
-0.5	Reciprocal square root
0	Logarithm
0.5	Square root
1	Original values
2	Square

These are useful interpretations of the Box-Cox power family.

The exact mathematical expression has a special case when lambda is zero.

The important point is that the transformation changes according to the value of lambda.

3.2 How does Power Transformation select lambda?

The process is:

1. Take the original feature.
2. Try different lambda values.
3. Transform the feature using each candidate value.
4. Examine how closely the resulting distribution resembles a Normal Distribution.
5. Select an appropriate lambda value.

The instructor mentions that the parameter is estimated using statistical techniques such as:

- Maximum Likelihood Estimation (MLE).
- Other statistical estimation methods.

You don't need to implement these mathematical optimization techniques manually when using Scikit-learn.

The library handles this calculation.

Remember: Power Transformation is a parametric transformation because it estimates a parameter (lambda) from the data.

---

4. Box-Cox Transformation

[Important concept]

Box-Cox Transformation is one of the two transformations available in Power Transformer.

It is used to make positively valued numerical data more normally distributed.

The mathematical formula is:

$$
X' =
\begin{cases}
\frac{X^\lambda-1}{\lambda}, & \lambda \neq 0\\

$$
\ln(X), & \lambda=0
\end{cases}
$$

Where:

- X = Original feature value.
- X' = Transformed feature value.
- λ = Estimated power parameter.
- $\ln$ = Natural logarithm.

4.1 Understanding the formula

Let's understand both cases.

Case 1: Lambda is not zero

Suppose:

$$
\lambda=2
$$

Then:

$$
X'=\frac{X^2-1}{2}
$$

If the original value is 3:

$$
X'=\frac{3^2-1}{2}
$$

$$
X'=\frac{9-1}{2}=4
$$

The transformed value is 4.

Case 2: Lambda is zero

When:

$$
\lambda=0
$$

The formula becomes:

$$
X'=\ln(X)
$$

For example, if:

$$
X=10
$$

Then:

$$
X'=\ln(10)\approx2.303
$$

This is why logarithmic transformation is considered a special case of Box-Cox transformation.

4.2 The biggest limitation of Box-Cox

Box-Cox Transformation has an important restriction:

[Restriction]

It only accepts strictly positive values.

That means:

$$
X>0
$$

It cannot directly handle:

- Zero values.
- Negative values.

For example:

$$
X=[2,5,10,20]
$$

This is valid.

But:

$$
X=[-2,0,5,10]
$$

\]

This is invalid for Box-Cox.

Why?

Because the transformation requires positive input values.

This limitation becomes important when working with real-world datasets.

---

5. Yeo-Johnson Transformation

Yeo-Johnson Transformation is another power transformation.

It was developed to overcome an important limitation of Box-Cox.

Remember:

Box-Cox only works with positive values.

But real-world datasets can contain:

- Positive values.
- Zero values.
- Negative values.

Yeo-Johnson can handle all three.

\[
X\in(-\infty,+\infty)
\]

This makes it more flexible.

5.1 Mathematical formula

Yeo-Johnson uses different mathematical expressions depending on whether the input is positive or negative.

\[
X'=
\begin{cases}
\frac{(X+1)^\lambda-1}{\lambda},
& X\geq0,\lambda\neq0\\
\ln(X+1),
& X\geq0,\lambda=0\\
-\frac{(1-X)^{2-\lambda}-1}{2-\lambda},
& X<0,\lambda\neq2\\
-\ln(1-X),
& X<0,\lambda=2

\end{cases}
\]

You do not need to memorize every part of this formula immediately.

Understand its main purpose:

- It handles positive values.
- It handles zero values.
- It handles negative values.
- It estimates the appropriate lambda parameter.

5.2 Box-Cox vs Yeo-Johnson

This is one of the most important comparisons in this lecture.

Property	Box-Cox	Yeo-Johnson
Positive values	Yes	Yes
Zero values	No	Yes
Negative values	No	Yes
Estimates lambda	Yes	Yes
Available in PowerTransformer	Yes	Yes

Practical conclusion: If your dataset contains only strictly positive values, you can try Box-Cox. If your dataset contains zero or negative values, Yeo-Johnson is the suitable choice between these two.

CHATGPT (Message 6)

6. Implementing Power Transformation in Scikit-learn

Now let's understand how we actually use Power Transformation in a Machine Learning project.

Scikit-learn provides a class called:

`PowerTransformer`

It is available inside:

from sklearn.preprocessing import PowerTransformer

The class supports two methods:

method='box-cox'

and

method='yeo-johnson'

By default, Scikit-learn uses Yeo-Johnson.

6.1 Creating a Power Transformer

First, create an object:

```
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer()
```

This creates a Power Transformer object.

By default:

```
pt = PowerTransformer(
method='yeo-johnson',
standardize=True
)
```

There are two important parameters.

1. method

Determines which transformation should be used.

- `'yeo-johnson'`
- `'box-cox'`

2. standardize

By default, it is `True`.

It means that after applying the power transformation, Scikit-learn also standardizes the transformed features.

Standardization generally converts a feature into values having approximately:

$$
\text{Mean}=0
$$

$$
\text{Standard Deviation}=1
$$

This is an additional operation, not the same thing as power transformation.

If you only want the power transformation without standardization:

```
pt = PowerTransformer(
method='yeo-johnson',
standardize=False
)
```

6.2 fit(), transform() and fit_transform()

These three methods are extremely important.

fit()

The model examines the training data and estimates the lambda parameter for each feature.
pt.fit(X_train)

transform()

It applies the previously learned transformation.
X_train_transformed = pt.transform(X_train)

fit_transform()

It performs both operations together.
X_train_transformed = pt.fit_transform(X_train)

For test data, use only:
X_test_transformed = pt.transform(X_test)

Why?

Because we should not calculate a new lambda using test data. The transformation parameters must be learned from training data and then reused on test data.

This prevents data leakage.

---

7. Practical Implementation: Concrete Strength Dataset

Now we will understand the practical experiment demonstrated in the lecture.

The instructor uses a Concrete Compressive Strength dataset.

The dataset contains information about different materials used to manufacture concrete.

The objective is to predict the compressive strength of concrete.

7.1 Understanding the dataset

The dataset contains input features such as:

Feature	Meaning
Cement	Amount of cement
Blast Furnace Slag	Amount of blast furnace slag
Fly Ash	Amount of fly ash

Water	Amount of water
Superplasticizer	Amount of superplasticizer
Coarse Aggregate	Amount of coarse aggregate
Fine Aggregate	Amount of fine aggregate
Age	Age of concrete
Concrete Strength	Target variable

There are eight input features and one output feature.

Therefore:

$$
X = 8\text{ input features}
$$

$$
y = 1\text{ target feature}
$$

This is a supervised learning problem.

More specifically, it is a regression problem.

7.2 Importing the required libraries

The instructor uses libraries such as Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn.

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PowerTransformer
from sklearn.metrics import r2_score
from sklearn.model_selection import cross_val_score
```

Each library has a specific responsibility:

- Pandas: Reading and manipulating datasets.
- NumPy: Numerical operations.
- Matplotlib: Creating graphs.
- Seaborn: Statistical visualizations.
- Scikit-learn: Building and evaluating machine learning models.

7.3 Loading the dataset

```
df = pd.read_csv('concrete_data.csv')
```

Separate the input features and target variable:

X = df.drop(columns=['Strength'])

y = df['Strength']

Here:

- X contains all eight input features.
- y contains concrete strength.

7.4 Checking missing values

Before applying transformations, inspect the dataset.

df.isnull().sum()

This checks how many missing values exist in every column.

The instructor finds that the dataset does not contain missing values.

This is useful because many preprocessing techniques require missing values to be handled before transformation.

7.5 Checking zero and negative values

Now comes a very important step.

Remember the restriction of Box-Cox:

\[
X>0
\]

Therefore, we should check whether any feature contains zero or negative values.

(X

Characteristics:

- Most observations are concentrated towards the left.
- A long tail extends towards the right.
- A few observations have very large values.

Left-skewed distribution (negative skewness)

Characteristics:

- Most observations are concentrated towards the right.
- A long tail extends towards the left.

The instructor observes that some features in the concrete dataset have highly skewed distributions.

For example, Age and some material-related features show noticeable distributional irregularities.

This is the reason for applying Power Transformation.

---

10. Applying Box-Cox Transformation

Now we apply the first transformation.

Create an object:
pt = PowerTransformer(method='box-cox')

Fit the transformer and transform the training data:
X_train_transformed = pt.fit_transform(X_train)

Transform the testing data:
X_test_transformed = pt.transform(X_test)

Now the original features have been transformed.

10.1 Understanding lambda values

One very important part of the lecture is that every input feature gets its own lambda value.

Suppose you have eight features:

$$
X=[X_1,X_2,\ldots,X_8]
$$

The transformer estimates:

$$
\lambda_1,\lambda_2,\ldots,\lambda_8
$$

This means that it does not necessarily apply the same power to every feature.

For example, imagine the following illustrative values:

Feature	Lambda
Cement	0.17
Blast Furnace Slag	0.23
Fly Ash	0.50

Water	0.80
Superplasticizer	0.30
Coarse Aggregate	1.20
Fine Aggregate	0.90
Age	0.10

These are illustrative values to explain the concept, not the exact output of the lecture.

You can inspect the actual estimated lambda values using:

pt.lambdas_

The output contains one lambda for each input feature.

This is important because different features have different distributions and therefore may require different transformations.

10.2 Training the model after Box-Cox

Now train another Linear Regression model.

lr = LinearRegression()

lr.fit(X_train_transformed, y_train)

Make predictions:

y_pred = lr.predict(X_test_transformed)

Evaluate:

r2_score(y_test, y_pred)

The instructor observes an improvement in the experimental results after applying Box-Cox.

However, the important thing is not simply obtaining a higher RÂ² score.

We should also check whether the transformed feature distributions have become closer to Normal Distributions.

---

11. Applying Yeo-Johnson Transformation

Now let's perform the same experiment using Yeo-Johnson.

Create the transformer:

pt1 = PowerTransformer(method='yeo-johnson')

Fit and transform:

X_train_transformed = pt1.fit_transform(X_train)

X_test_transformed = pt1.transform(X_test)

Train the model:

lr = LinearRegression()

lr.fit(X_train_transformed, y_train)

Predict:

y_pred = lr.predict(X_test_transformed)

Evaluate:

r2_score(y_test, y_pred)

The instructor repeats the same experiment using Yeo-Johnson.

The main objective is to compare:

1. Original data.
2. Box-Cox transformed data.
3. Yeo-Johnson transformed data.

The instructor observes that both transformations improve the experimental results, with Yeo-Johnson performing better in the reported comparison.

However, this does not mean Yeo-Johnson will always outperform Box-Cox. Performance depends on the dataset, model, target relationship and evaluation method.

CHATGPT (Message 7)

Lecture 31: Power Transformation

Machine Learning — Feature Transformation | Complete study notes

[Transcript received and covered]

This chapter is based on the complete transcript you uploaded. I've organized the lecture into a proper learning chapter, explained the concepts in simple English, and expanded the technical points so that you can study without an internet connection.

The lecture covers three important things:

1. Power Transformation
2. Box-Cox Transformation
3. Yeo-Johnson Transformation

It also demonstrates these transformations using a real-world concrete strength dataset and Linear Regression.

---

1. Why do we need mathematical transformations?

Before understanding Power Transformation, we need to understand why mathematical transformations are required in Machine Learning.

Suppose you have a dataset with different input features:

Cement	Water	Age	Concrete strength
250	180	28	35
350	200	90	52
150	160	7	20
400	190	56	60

Our objective is to predict concrete strength using the input features.

This is a **Regression Problem** because the output (concrete strength) is a continuous numerical value.

We can train a Linear Regression model on this dataset.

However, there is one possible problem.

The input features may not follow a Normal Distribution.

For example:
- Some features may have most values concentrated around smaller numbers.
- A few observations may have extremely large values.
- The distribution may be heavily skewed towards the right or left.

These distributions can affect the performance of certain machine learning algorithms.

This is where mathematical transformations become useful.

1.1 What is a mathematical transformation?

A mathematical transformation means changing the values of a feature using a mathematical function.

For example, suppose a feature contains:

$$
X=[1,4,9,16,25]
$$

If we apply a square-root transformation:

$$
X'=\sqrt{X}
$$

The transformed values become:

$$
X'=[1,2,3,4,5]
$$

Notice that the original values have changed, but the observations are still in the same order.

We have simply represented the data differently.

Why do we do this?

To make the distribution of a feature closer to a Normal Distribution.

This is particularly useful when working with algorithms that benefit from approximately normally distributed features.

Examples include:
- Linear Regression (in certain modelling situations).
- Logistic Regression.
- Some statistical models that make distributional assumptions.

An important distinction: Normality of input features is not a universal requirement for Linear Regression or Logistic Regression. Transformations can help, but they do not automatically guarantee a better model.

---

2. Types of Mathematical Transformations

In the previous lecture, the instructor discussed Function Transformer.

There are two major categories in this part of the syllabus.

1. Function Transformer

It allows us to apply mathematical functions such as:

- Log transformation
- Square-root transformation
- Reciprocal transformation

2. Power Transformer

It provides two important transformations:

- Box-Cox Transformation
- Yeo-Johnson Transformation

The main difference is that Power Transformer can estimate the appropriate power parameter from the data instead of requiring us to manually choose a mathematical function.

---

3. What is Power Transformation?

Power Transformation is a family of mathematical transformations used to make data more closely resemble a Normal Distribution.

It automatically determines an appropriate power parameter, usually represented by the Greek letter lambda:

$$
\lambda
$$

The important question is:

Which power should we apply to our feature?

Should we use:

$$
X^2
$$

Or:

$$
X^3
$$

Or:

$$
\sqrt X
$$

Or:

$$
\log(X)
$$

We usually don't know which transformation will work best.

Power Transformation attempts to find a suitable value of lambda based on the data.

3.1 Understanding Lambda (λ)

Lambda represents the power parameter used in the transformation.

For example:

Lambda	Approximate transformation
-2	Reciprocal square

-1	Reciprocal
-0.5	Reciprocal square root
0	Logarithm
0.5	Square root
1	Original values
2	Square

These are useful interpretations of the Box-Cox power family.

The exact mathematical expression has a special case when lambda is zero.

The important point is that the transformation changes according to the value of lambda.

3.2 How does Power Transformation select lambda?

The process is:

1. Take the original feature.
2. Try different lambda values.
3. Transform the feature using each candidate value.
4. Examine how closely the resulting distribution resembles a Normal Distribution.
5. Select an appropriate lambda value.

The instructor mentions that the parameter is estimated using statistical techniques such as:

- Maximum Likelihood Estimation (MLE).
- Other statistical estimation methods.

You don't need to implement these mathematical optimization techniques manually when using Scikit-learn.

The library handles this calculation.

Remember: Power Transformation is a parametric transformation because it estimates a parameter (lambda) from the data.

---

4. Box-Cox Transformation

[Important concept]

Box-Cox Transformation is one of the two transformations available in Power Transformer.

It is used to make positively valued numerical data more normally distributed.

The mathematical formula is:

$$
X' =
\begin{cases}
\frac{X^\lambda-1}{\lambda}, & \lambda \neq 0\\

$$
\ln(X), & \lambda=0
\end{cases}
$$

Where:

- X = Original feature value.
- X' = Transformed feature value.
- λ = Estimated power parameter.
- $\ln$ = Natural logarithm.

4.1 Understanding the formula

Let's understand both cases.

Case 1: Lambda is not zero

Suppose:

$$
\lambda=2
$$

Then:

$$
X'=\frac{X^2-1}{2}
$$

If the original value is 3:

$$
X'=\frac{3^2-1}{2}
$$

$$
X'=\frac{9-1}{2}=4
$$

The transformed value is 4.

Case 2: Lambda is zero

When:

$$
\lambda=0
$$

The formula becomes:

$$
X'=\ln(X)
$$

For example, if:

$$
X=10
$$

Then:

$$
X'=\ln(10)\approx2.303
$$

This is why logarithmic transformation is considered a special case of Box-Cox transformation.

4.2 The biggest limitation of Box-Cox

Box-Cox Transformation has an important restriction:

[Restriction]

It only accepts strictly positive values.

That means:

$$
X>0
$$

It cannot directly handle:

- Zero values.
- Negative values.

For example:

$$
X=[2,5,10,20]
$$

This is valid.

But:

$$
X=[-2,0,5,10]
$$

\]

This is invalid for Box-Cox.

Why?

Because the transformation requires positive input values.

This limitation becomes important when working with real-world datasets.

---

5. Yeo-Johnson Transformation

Yeo-Johnson Transformation is another power transformation.

It was developed to overcome an important limitation of Box-Cox.

Remember:

Box-Cox only works with positive values.

But real-world datasets can contain:

- Positive values.
- Zero values.
- Negative values.

Yeo-Johnson can handle all three.

\[
X\in(-\infty,+\infty)
\]

This makes it more flexible.

5.1 Mathematical formula

Yeo-Johnson uses different mathematical expressions depending on whether the input is positive or negative.

\[
X'=
\begin{cases}
\frac{(X+1)^\lambda-1}{\lambda},
& X\geq0,\lambda\neq0\\
\ln(X+1),
& X\geq0,\lambda=0\\
-\frac{(1-X)^{2-\lambda}-1}{2-\lambda},
& X0
\]

Therefore, we should check whether any feature contains zero or negative values.

(X

Characteristics:

- Most observations are concentrated towards the left.
- A long tail extends towards the right.
- A few observations have very large values.

Left-skewed distribution (negative skewness)

Characteristics:

- Most observations are concentrated towards the right.
- A long tail extends towards the left.

The instructor observes that some features in the concrete dataset have highly skewed distributions.

For example, Age and some material-related features show noticeable distributional irregularities.

This is the reason for applying Power Transformation.

---

10. Applying Box-Cox Transformation

Now we apply the first transformation.

Create an object:

pt = PowerTransformer(method='box-cox')

Fit the transformer and transform the training data:

X_train_transformed = pt.fit_transform(X_train)

Transform the testing data:

X_test_transformed = pt.transform(X_test)

Now the original features have been transformed.

10.1 Understanding lambda values

One very important part of the lecture is that every input feature gets its own lambda value.

Suppose you have eight features:

\[
X=[X_1,X_2,\ldots,X_8]

\]

The transformer estimates:

\[
\lambda_1,\lambda_2,\ldots,\lambda_8
\]

This means that it does not necessarily apply the same power to every feature.

For example, imagine the following illustrative values:

Feature	Lambda
Cement	0.17
Blast Furnace Slag	0.23
Fly Ash	0.50
Water	0.80
Superplasticizer	0.30
Coarse Aggregate	1.20
Fine Aggregate	0.90
Age	0.10

These are illustrative values to explain the concept, not the exact output of the lecture.

You can inspect the actual estimated lambda values using:

```
pt.lambdas_
```

The output contains one lambda for each input feature.

This is important because different features have different distributions and therefore may require different transformations.

10.2 Training the model after Box-Cox

Now train another Linear Regression model.

```
lr = LinearRegression()

lr.fit(X_train_transformed, y_train)
```

Make predictions:

```
y_pred = lr.predict(X_test_transformed)
```

Evaluate:

```
r2_score(y_test, y_pred)
```

The instructor observes an improvement in the experimental results after applying Box-Cox.

However, the important thing is not simply obtaining a higher R^2 score.

We should also check whether the transformed feature distributions have become closer to Normal Distributions.

---

11. Applying Yeo-Johnson Transformation

Now let's perform the same experiment using Yeo-Johnson.

Create the transformer:

```
pt1 = PowerTransformer(method='yeo-johnson')
```

Fit and transform:

```
X_train_transformed = pt1.fit_transform(X_train)

X_test_transformed = pt1.transform(X_test)
```

Train the model:

```
lr = LinearRegression()

lr.fit(X_train_transformed, y_train)
```

Predict:

```
y_pred = lr.predict(X_test_transformed)
```

Evaluate:

```
r2_score(y_test, y_pred)
```

The instructor repeats the same experiment using Yeo-Johnson.

The main objective is to compare:

1. Original data.
2. Box-Cox transformed data.
3. Yeo-Johnson transformed data.

The instructor observes that both transformations improve the experimental results, with Yeo-Johnson performing better in the reported comparison.

However, this does not mean Yeo-Johnson will always outperform Box-Cox. Performance depends on the dataset, model, target relationship and evaluation method.

---

12. Comparing Distributions Before and After Transformation

This is one of the most useful practical parts of the lecture.

After applying Power Transformation, the instructor compares the original feature distributions with the transformed distributions.

Why?

Because the main objective of Power Transformation is to make the data more closely resemble a Normal Distribution.

The comparison is performed using histograms.

Let's understand what we should observe.

Before transformation	After transformation
Distribution may be highly skewed	Distribution may become more symmetric
Long tails may be present	Tails may become less pronounced
Values may be concentrated in one region	Values may become more evenly distributed
Feature may be far from Normal	Feature may become closer to Normal

Important: A transformation does not guarantee a perfectly Normal Distribution.

Some features may improve significantly, while others may show very little improvement.

The instructor observes that the Age feature shows particularly noticeable improvement, while some other features remain relatively similar.

This is expected because each feature has a different original distribution.

13. Final Comparison of All Three Experiments

The instructor essentially performs three experiments.

Experiment 1: Original data

Original dataset

Linear Regression

Evaluate R^2

No mathematical transformation is applied.

Experiment 2: Box-Cox

Original dataset

Box-Cox Transformation

Linear Regression

Evaluate R^2

The features are transformed using estimated lambda values.

Experiment 3: Yeo-Johnson

Original dataset

Yeo-Johnson Transformation

Linear Regression

Evaluate R^2

The features are transformed using Yeo-Johnson.

The instructor compares the R^2 scores and feature distributions.

The important conclusion is that Power Transformation can improve the performance of Linear Regression by changing the distribution of input features.

The transcript's numerical results are not sufficiently clear in a few places to reproduce every exact score reliably, so I have not invented a complete numerical comparison.

---

14. When should we use Power Transformation?

Suppose you are working on a Machine Learning project.

You have numerical features, and after plotting their distributions, you discover that some features are highly skewed.

You are using algorithms such as:

- Linear Regression.
- Logistic Regression.

- Other models that can benefit from more suitable feature distributions.

In this situation, you can try Power Transformation.

The workflow should be:

1. Inspect the original feature distributions.
2. Check for missing values.
3. Check whether zero or negative values exist.
4. Apply Box-Cox or Yeo-Johnson.
5. Compare distributions before and after transformation.
6. Train your model.
7. Evaluate its performance using appropriate validation.

The instructor recommends experimenting with different transformations and choosing the one that gives better results for your project.

14.1 Does every Machine Learning algorithm need Power Transformation?

No.

This is important.

Power Transformation is not compulsory.

For example:

Linear Regression

Can benefit from transformations when they improve the relationship between features and the target or address modelling issues.

Logistic Regression

May benefit from better-behaved numerical features.

Decision Trees

Generally do not require normally distributed features.

Random Forest

Usually does not require normally distributed features.

XGBoost

Also generally does not require normally distributed input features.

Therefore, always evaluate whether the transformation actually helps.

15. Important Practical Concepts You Should Know

These are some additional implementation details that will help you avoid mistakes in real projects.

15.1 Power Transformation is applied feature-wise

Suppose your dataset contains:

X.shape = (1000, 8)

This means:

- 1000 rows.
- 8 columns.

When you apply:

pt = PowerTransformer()

X_transformed = pt.fit_transform(X)

Scikit-learn estimates a separate lambda for each column.

It does not calculate one common lambda for all eight columns.

15.2 Power Transformation and Standardization are different

Remember the default:

PowerTransformer(standardize=True)

The transformer first applies the power transformation and then standardizes the resulting features.

If you want to separate the two operations:

pt = PowerTransformer(standardize=False)

This performs only the power transformation.

15.3 Never fit the transformer separately on test data

Incorrect:

X_train = pt.fit_transform(X_train)

X_test = pt.fit_transform(X_test)

Why is this wrong?

Because the training and testing data would have different estimated lambda values.

Correct:

```
X_train = pt.fit_transform(X_train)

X_test = pt.transform(X_test)
```

The test data must use the transformation learned from training data.

15.4 Use a Pipeline in real projects

A better approach is to use a Pipeline.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression

pipe = Pipeline([
('power', PowerTransformer(
method='yeo-johnson'
)),
('model', LinearRegression())
])

pipe.fit(X_train, y_train)

y_pred = pipe.predict(X_test)
```

Why is this useful?

- The transformation is fitted automatically.
- The same transformation is applied during prediction.
- It helps prevent data leakage.
- It makes your code cleaner.
- It works well with cross-validation.

For real-world projects, this is generally preferable to manually transforming training and testing data.

---

16. Complete Code Revision

Here is a consolidated implementation of the three experiments.

You can save this code and revise it later.

```
import numpy as np
import pandas as pd

from sklearn.model_selection import (
train_test_split,
cross_val_score
)

from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import r2_score
from sklearn.pipeline import Pipeline
```

Load dataset

```
df = pd.read_csv('concrete_data.csv')
```

Separate X and y

```
X = df.drop(columns=['Strength'])
y = df['Strength']
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```

Experiment 1: Original data

```
lr = LinearRegression()

lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

print("Original R2:", r2_score(y_test, y_pred))
```

Experiment 2: Box-Cox

Only use if all training feature values are strictly positive.

```
box_pipe = Pipeline([
('transform', PowerTransformer(
method='box-cox'
)),
('model', LinearRegression())
])

box_pipe.fit(X_train, y_train)

y_pred_box = box_pipe.predict(X_test)

print("Box-Cox R2:", r2_score(y_test, y_pred_box))
```

Experiment 3: Yeo-Johnson

```
yeo_pipe = Pipeline([
('transform', PowerTransformer(
```

```
method='yeo-johnson'
)),
('model', LinearRegression())
])

yeo_pipe.fit(X_train, y_train)

y_pred_yeo = yeo_pipe.predict(X_test)

print("Yeo-Johnson R2:", r2_score(y_test, y_pred_yeo))
```

Note: This is a consolidated implementation of the lecture's experiments, with a Pipeline added for safer preprocessing. It assumes the dataset has already been cleaned and that the Box-Cox input features are strictly positive.

---

17. Quick Revision: Everything You Need to Remember

Power Transformation
Definition: A mathematical transformation used to make numerical features more closely resemble a Normal Distribution.

Parameter: Lambda (λ).

Main methods:
- Box-Cox
- Yeo-Johnson

Box-Cox

- Only accepts strictly positive values.
- Does not accept zero.
- Does not accept negative values.
- Estimates lambda automatically.

Yeo-Johnson

- Accepts positive values.
- Accepts zero.
- Accepts negative values.
- Estimates lambda automatically.

Scikit-learn

from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer(
method='yeo-johnson'
)

X_train = pt.fit_transform(X_train)
X_test = pt.transform(X_test)

Default settings:
- Method: `yeo-johnson`
- Standardize: `True`

Inspect lambda values:

pt.lambdas_

18. Test Yourself

Try answering these questions without looking at the notes.

{@body const questions = [{q:"What is the main purpose of Power Transformation?",a:["To remove missing values","To make feature distributions closer to Normal","To convert categorical data into numbers","To select important features"],c:1,e:"Power Transformation changes the shape of numerical feature distributions."},{q:"Which transformation accepts negative values?",a:["Box-Cox","Log transformation","Yeo-Johnson","Square-root transformation"],c:2,e:"Yeo-Johnson supports positive, zero and negative values."},{q:"What does lambda represent?",a:["Number of rows","Power parameter","Target variable","Number of features"],c:1,e:"Lambda controls the mathematical form of the power transformation."},{q:"What is the default method in Scikit-learn PowerTransformer?",a:["Box-Cox","Yeo-Johnson","Log","Square root"],c:1,e:"Scikit-learn uses Yeo-Johnson by default."},{q:"What should you use on test data after fitting a transformer on training data?",a:["fit_transform()","fit()","transform()","fit_predict()"],c:2,e:"Use transform() so the test data uses the parameters learned from training data."},{q:"Does every machine learning algorithm require normally distributed input features?",a:["Yes","No"],c:1,e:"Tree-based algorithms generally do not require normally distributed features."}]}
{@body const [answers,setAnswers] = DIL.useState({})}
{@body const [submitted,setSubmitted] = DIL.useState(false)}

Lecture 31 quiz
[{Object.keys(answers).length}/{questions.length} answered]

{#each questions as q,i}

{i+1}. {q.q}
setAnswers(p=>({...p,[i]:Number(v)}))}>
{#each q.a as option,j}
{option}
{/each}

{#if submitted}

{#if answers[i]===q.c}

Correct
{:else}

Incorrect — {q.a[q.c]}
{/if}

{q.e}

{/if}

{#if i!==questions.length-1}

{/if}
{/each}
{#if submitted}
{questions.filter((q,i)=>answers[i]===q.c).length}/{questions.length} correct
{setAnswers({});setSubmitted(false)}}>Try again
{:else}
setSubmitted(true)}>Submit answers
{/if}

---

19. Final Understanding

If you remember only one thing from this lecture, remember this:

Power Transformation helps us change the distribution of numerical features.

It automatically estimates a suitable power parameter called lambda.

It provides two methods:

- Box-Cox: For strictly positive data.
- Yeo-Johnson: For positive, zero and negative data.

We can use these transformations in Scikit-learn through `PowerTransformer`.

Finally, compare the results using proper validation. Do not assume that a more Normal-looking feature will always produce a better predictive model.

Lecture 31 is covered.

Next, we'll continue with Lecture 32, following the same detailed, offline-study format.

CHATGPT (Message 8)

12. Comparing Distributions Before and After Transformation

This is one of the most useful practical parts of the lecture.

After applying Power Transformation, the instructor compares the original feature distributions with the transformed distributions.

Why?

Because the main objective of Power Transformation is to make the data more closely resemble a Normal Distribution.

The comparison is performed using histograms.

Let's understand what we should observe.

Before transformation	After transformation
Distribution may be highly skewed	Distribution may become more symmetric
Long tails may be present	Tails may become less pronounced
Values may be concentrated in one region	Values may become more evenly distributed
Feature may be far from Normal	Feature may become closer to Normal

Important: A transformation does not guarantee a perfectly Normal Distribution.

Some features may improve significantly, while others may show very little improvement.

The instructor observes that the Age feature shows particularly noticeable improvement, while some other features remain relatively similar.

This is expected because each feature has a different original distribution.

13. Final Comparison of All Three Experiments

The instructor essentially performs three experiments.

Experiment 1: Original data

Original dataset

Linear Regression

Evaluate R^2

No mathematical transformation is applied.

Experiment 2: Box-Cox

Original dataset

Box-Cox Transformation

Linear Regression

Evaluate R^2

The features are transformed using estimated lambda values.

Experiment 3: Yeo-Johnson

Original dataset

Yeo-Johnson Transformation

Linear Regression

Evaluate R^2

The features are transformed using Yeo-Johnson.

The instructor compares the R^2 scores and feature distributions.

The important conclusion is that Power Transformation can improve the performance of Linear Regression by changing the distribution of input features.

The transcript's numerical results are not sufficiently clear in a few places to reproduce every exact score reliably, so I have not invented a complete numerical comparison.

---

14. When should we use Power Transformation?

Suppose you are working on a Machine Learning project.

You have numerical features, and after plotting their distributions, you discover that some features are highly skewed.

You are using algorithms such as:

- Linear Regression.
- Logistic Regression.
- Other models that can benefit from more suitable feature distributions.

In this situation, you can try Power Transformation.

The workflow should be:

1. Inspect the original feature distributions.
2. Check for missing values.
3. Check whether zero or negative values exist.
4. Apply Box-Cox or Yeo-Johnson.
5. Compare distributions before and after transformation.
6. Train your model.
7. Evaluate its performance using appropriate validation.

The instructor recommends experimenting with different transformations and choosing the one that gives better results for your project.

14.1 Does every Machine Learning algorithm need Power Transformation?

No.

This is important.

Power Transformation is not compulsory.

For example:

Linear Regression

Can benefit from transformations when they improve the relationship between features and the target or address modelling issues.

Logistic Regression

May benefit from better-behaved numerical features.

Decision Trees

Generally do not require normally distributed features.

Random Forest

Usually does not require normally distributed features.

XGBoost

Also generally does not require normally distributed input features.

Therefore, always evaluate whether the transformation actually helps.

15. Important Practical Concepts You Should Know

These are some additional implementation details that will help you avoid mistakes in real projects.

15.1 Power Transformation is applied feature-wise

Suppose your dataset contains:

X.shape = (1000, 8)

This means:

- 1000 rows.
- 8 columns.

When you apply:

pt = PowerTransformer()

X_transformed = pt.fit_transform(X)

Scikit-learn estimates a separate lambda for each column.

It does not calculate one common lambda for all eight columns.

15.2 Power Transformation and Standardization are different

Remember the default:

PowerTransformer(standardize=True)

The transformer first applies the power transformation and then standardizes the resulting features.

If you want to separate the two operations:

pt = PowerTransformer(standardize=False)

This performs only the power transformation.

15.3 Never fit the transformer separately on test data

Incorrect:

X_train = pt.fit_transform(X_train)

X_test = pt.fit_transform(X_test)

Why is this wrong?

Because the training and testing data would have different estimated lambda values.

Correct:

```
X_train = pt.fit_transform(X_train)

X_test = pt.transform(X_test)
```

The test data must use the transformation learned from training data.

15.4 Use a Pipeline in real projects

A better approach is to use a Pipeline.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression

pipe = Pipeline([
('power', PowerTransformer(
method='yeo-johnson'
)),
('model', LinearRegression())
])

pipe.fit(X_train, y_train)

y_pred = pipe.predict(X_test)
```

Why is this useful?

- The transformation is fitted automatically.
- The same transformation is applied during prediction.
- It helps prevent data leakage.
- It makes your code cleaner.
- It works well with cross-validation.

For real-world projects, this is generally preferable to manually transforming training and testing data.

---

16. Complete Code Revision

Here is a consolidated implementation of the three experiments.

You can save this code and revise it later.

```
import numpy as np
import pandas as pd

from sklearn.model_selection import (
train_test_split,
```

```
cross_val_score
)

from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.pipeline import Pipeline
```

Load dataset

```
df = pd.read_csv('concrete_data.csv')
```

Separate X and y

```
X = df.drop(columns=['Strength'])
y = df['Strength']
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```

Experiment 1: Original data

```
lr = LinearRegression()

lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

print("Original R2:", r2_score(y_test, y_pred))
```

Experiment 2: Box-Cox

Only use if all training feature values are strictly positive.

```
box_pipe = Pipeline([
('transform', PowerTransformer(
method='box-cox'
)),
('model', LinearRegression())
])

box_pipe.fit(X_train, y_train)

y_pred_box = box_pipe.predict(X_test)

print("Box-Cox R2:", r2_score(y_test, y_pred_box))
```

Experiment 3: Yeo-Johnson

```
yeo_pipe = Pipeline([
('transform', PowerTransformer(
method='yeo-johnson'
)),
('model', LinearRegression())
])

yeo_pipe.fit(X_train, y_train)

y_pred_yeo = yeo_pipe.predict(X_test)

print("Yeo-Johnson R2:", r2_score(y_test, y_pred_yeo))
```

Note: This is a consolidated implementation of the lecture's experiments, with a Pipeline added for safer preprocessing. It assumes the dataset has already been cleaned and that the Box-Cox input features are strictly positive.

---

17. Quick Revision: Everything You Need to Remember

Power Transformation
Definition: A mathematical transformation used to make numerical features more closely resemble a Normal Distribution.

Parameter: Lambda (λ).

Main methods:
- Box-Cox
- Yeo-Johnson

Box-Cox

- Only accepts strictly positive values.
- Does not accept zero.
- Does not accept negative values.
- Estimates lambda automatically.

Yeo-Johnson

- Accepts positive values.
- Accepts zero.
- Accepts negative values.
- Estimates lambda automatically.

Scikit-learn

```
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer(
method='yeo-johnson'
)

X_train = pt.fit_transform(X_train)
X_test = pt.transform(X_test)
```

Default settings:
- Method: `yeo-johnson`
- Standardize: `True`

Inspect lambda values:

```
pt.lambdas_
```

18. Test Yourself

Try answering these questions without looking at the notes.

{@body const questions = [{q:"What is the main purpose of Power Transformation?",a:["To remove missing values","To make feature distributions closer to Normal","To convert categorical data into numbers","To select important features"],c:1,e:"Power Transformation changes the shape of numerical feature distributions."},{q:"Which transformation accepts negative values?",a:["Box-Cox","Log transformation","Yeo-Johnson","Square-root transformation"],c:2,e:"Yeo-Johnson supports positive, zero and negative values."},{q:"What does lambda represent?",a:["Number of rows","Power parameter","Target variable","Number of features"],c:1,e:"Lambda controls the mathematical form of the power transformation."},{q:"What is the default method in Scikit-learn PowerTransformer?",a:["Box-Cox","Yeo-Johnson","Log","Square root"],c:1,e:"Scikit-learn uses Yeo-Johnson by default."},{q:"What should you use on test data after fitting a transformer on training data?",a:["fit_transform()","fit()","transform()","fit_predict()"],c:2,e:"Use transform() so the test data uses the parameters learned from training data."},{q:"Does every machine learning algorithm require normally distributed input features?",a:["Yes","No"],c:1,e:"Tree-based algorithms generally do not require normally distributed features."}]}
{@body const [answers,setAnswers] = DIL.useState({})}
{@body const [submitted,setSubmitted] = DIL.useState(false)}

Lecture 31 quiz
[{Object.keys(answers).length}/{questions.length} answered]

{#each questions as q,i}

{i+1}. {q.q}
setAnswers(p=>({...p,[i]:Number(v)}))}>
{#each q.a as option,j}

{option}
{/each}

{#if submitted}

{#if answers[i]===q.c}

Correct
{:else}

Incorrect — {q.a[q.c]}
{/if}

{q.e}

{/if}

{#if i!==questions.length-1}

{/if}
{/each}
{#if submitted}
{questions.filter((q,i)=>answers[i]===q.c).length}/{questions.length} correct
{setAnswers({});setSubmitted(false)}}>Try again
{:else}
setSubmitted(true)}>Submit answers
{/if}

---

19. Final Understanding

If you remember only one thing from this lecture, remember this:

Power Transformation helps us change the distribution of numerical features.

It automatically estimates a suitable power parameter called lambda.

It provides two methods:

- Box-Cox: For strictly positive data.
- Yeo-Johnson: For positive, zero and negative data.

We can use these transformations in Scikit-learn through `PowerTransformer`.

Finally, compare the results using proper validation. Do not assume that a more Normal-looking feature will always produce a better predictive model.

Lecture 31 is covered.

Next, we'll continue with Lecture 32, following the same detailed, offline-study format.

USER (Message 9)

Go ahead for 32

[Attachment: Binning and Binarization Discretization Quantile Binning KMeans Binning.txt (75658 bytes)]

CHATGPT (Message 10)

Lecture 32: Binning and Binarization

Machine Learning — Feature Engineering | Complete offline study notes

[Complete transcript covered]
[Lecture 32]

Topics covered in this lecture:

1. Discretization (Binning)
2. Equal Width Binning
3. Equal Frequency Binning (Quantile Binning)
4. K-Means Binning
5. Custom Binning
6. Binarization
7. Scikit-learn implementation
8. Practical implementation using the Titanic dataset

The objective of this chapter is to understand how we convert numerical features into categorical or binary features, why we do it, and how to implement it in Scikit-learn.

PART 1 — DISCRETIZATION (BINNING)

1. Why do we need to convert numerical data into categorical data?

Suppose you are working on a Machine Learning project.

You have a dataset containing information about different mobile applications.

One column contains the number of downloads.

Application	Downloads
App A	500
App B	5,000
App C	50,000
App D	500,000

App E	5,000,000
App F	50,000,000
App G	100,000,000

Notice something.

Some applications have very few downloads, while some applications have millions of downloads.

The numerical values have a very large range.

Sometimes, working with the exact numerical values is not the most useful representation for a particular problem.

Instead, we could create categories:

Downloads	Category
500	Low downloads
5,000	Low downloads
50,000	Low downloads
500,000	Medium downloads
5,000,000	High downloads
50,000,000	Very high downloads
100,000,000	Very high downloads

Now the original numerical feature has been converted into categories.

This is the basic idea behind **Discretization**.

The instructor uses this type of example to explain why converting numerical data into categorical data can sometimes provide a better representation for a particular problem.

2. What is Discretization?

[Important definition]

Discretization is the process of converting continuous numerical values into a limited number of discrete intervals or categories.

It is also called **Binning**.

Let's understand the definition.

Suppose we have:

$$
X=[2,5,8,12,15,18,22,25,28]
$$

These are numerical values.

We want to divide them into three intervals:

- Bin 1: 0–10
- Bin 2: 10–20
- Bin 3: 20–30

Now each observation belongs to one interval.

Original value	Assigned bin
2	0–10
5	0–10
8	0–10
12	10–20
15	10–20
18	10–20
22	20–30
25	20–30
28	20–30

Instead of working with the exact values, we now work with their corresponding intervals.

This is called Binning.

2.1 Understanding bins

A bin is simply an interval or group into which numerical values are placed.

For example:

\[
[0,10),[10,20),[20,30]
\]

These are three bins.

The first bin contains values from 0 up to, but not including, 10.

The second bin contains values from 10 up to, but not including, 20.

The third bin contains values from 20 to 30.

The exact boundary convention depends on the implementation.

2.2 How does binning work?

The process is straightforward:

1. Take a numerical feature.

2. Decide how many bins you want.
3. Create intervals.
4. Assign every observation to the appropriate interval.
5. Represent the interval using a category or numerical code.

You can think of it as converting a continuous range into a limited number of groups.

2.3 Why do we use Discretization?

The instructor discusses three important benefits.

1. Handling outliers

Suppose your age feature contains:

$$
[18,20,22,25,28,30,35,40,150]
$$

The value 150 is an extreme value.

If we create bins:

- 0–20
- 20–40
- 40–60
- 60–80
- 80–100
- 100–160

The extreme value gets assigned to the final bin.

The model works with its bin representation rather than treating 150 as a separate extreme numerical value.

This can reduce the influence of extreme values in some modelling approaches.

However, remember that binning does not remove an outlier. It only changes its representation.

2. Improving the spread of data

Sometimes numerical observations are concentrated in a small region, while a few observations are spread over a much larger range.

Depending on the binning method, we can create categories that distribute observations more evenly.

We will understand this properly when we study Quantile Binning.

3. Simplifying the representation

Instead of handling hundreds or thousands of different numerical values, we can work with a smaller number of categories.

For example:

- Age 18
- Age 19
- Age 20
- Age 21
- Age 22

Can be represented using:

- Young
- Adult
- Middle-aged
- Senior

The categories can be easier to interpret in certain applications.

PART 2 — TYPES OF BINNING

3. Classification of Discretization techniques

The instructor divides binning into two main categories.

Discretization techniques

Discretization / Binning

Unsupervised

1. Equal Width Binning

2. Equal Frequency Binning

3. K-Means Binning

Supervised

1. Decision Tree Binning

The instructor focuses on the three unsupervised techniques and custom binning.

Decision Tree Binning is mentioned but not explained in detail in this lecture.

Before going further, understand the difference.

Unsupervised Binning: The binning process does not use the target variable.

Supervised Binning: The target variable is used when determining how the numerical feature should be divided.

For example, if you are predicting whether a customer will purchase a product, supervised binning can use the purchase outcome while deciding the bin boundaries.

PART 3 — EQUAL WIDTH BINNING

4. What is Equal Width Binning?

Equal Width Binning is a discretization technique in which every bin has the same numerical width.

The instructor also calls it:

- Uniform Binning
- Equal Width Discretization

The number of bins is decided by us.

Let's understand this with an example.

Suppose we have:

$$
X=[2,5,8,12,15,18,22,25,28]
$$

The minimum value is:

$$
X_{\min}=2
$$

The maximum value is:

$$
X_{\max}=28
$$

Suppose we want three bins.

The formula for bin width is:

$$
\text{Bin Width}=
\frac{\text{Maximum}-\text{Minimum}}
{\text{Number of bins}}
$$

Substituting the values:

$$
=\frac{28-2}{3}
$$

$$
=\frac{26}{3}
$$

$$
=8.67
$$

Therefore, each bin will have approximately 8.67 units of width.

4.1 Creating the intervals

Starting from the minimum value, we create three intervals.

Bin	Interval
Bin 1	2–10.67
Bin 2	10.67–19.33
Bin 3	19.33–28

Notice something important:

All three intervals have approximately the same width.

That is why it is called Equal Width Binning.

4.2 Assigning observations

Our original data:

\[
[2,5,8,12,15,18,22,25,28]
\]

After assigning the values:

Value	Bin
2	1
5	1
8	1
12	2
15	2
18	2
22	3
25	3
28	3

The distribution of observations across bins is:

Bin	Number of observations
Bin 1	3
Bin 2	3
Bin 3	3

In this example, every bin happens to contain three observations.

But that is not guaranteed by Equal Width Binning.

If your data is highly skewed, one bin might contain 100 observations while another contains only 2.

This is a very important distinction.

4.3 Visual understanding

Here, the intervals have equal width, but the number of observations in each interval can differ.

4.4 Advantages of Equal Width Binning

1. Simple to understand

The formula is straightforward.

2. Easy to implement

You only need the minimum, maximum and number of bins.

3. Helps limit extreme numerical values

Very large values can be grouped into the final interval.

4.5 Disadvantages of Equal Width Binning

The biggest limitation is that it does not guarantee equal numbers of observations in each bin.

For example:

Suppose 900 observations are between 0 and 20, while only 100 observations are between 80 and 100.

If we divide the range into five equal-width bins, the first bins may contain most of the observations.

The last bins may contain very few observations.

Therefore, Equal Width Binning does not necessarily make the distribution of observations across bins uniform.

CHATGPT (Message 11)

PART 4 — EQUAL FREQUENCY BINNING (QUANTILE BINNING)

5. What is Equal Frequency Binning?

[Very important]

Equal Frequency Binning is another method of discretization.

It is also called:

- Quantile Binning
- Equal Frequency Discretization

In Equal Width Binning, every bin has the same width.

But in Equal Frequency Binning, every bin contains approximately the same number of observations.

This is the biggest difference.

Let's understand it properly.

5.1 Understanding the concept

Suppose you have 100 students and their marks.

You want to divide them into five bins.

In Equal Frequency Binning, each bin should contain approximately:

$$
\frac{100}{5}=20
$$

students.

Therefore:

- Bin 1 contains 20 students.
- Bin 2 contains 20 students.
- Bin 3 contains 20 students.
- Bin 4 contains 20 students.
- Bin 5 contains 20 students.

Notice that we are not trying to make the numerical width of every interval equal.

We are trying to make the number of observations equal.

5.2 A numerical example

Suppose we have the following sorted data:

$$
X=[2,3,5,7,9,12,15,18,22,25]
$$

There are 10 observations.

Suppose we want five bins.

Each bin should contain:

$$
\frac{10}{5}=2
$$

observations.

We can create the following bins:

Bin	Values	Frequency
Bin 1	2, 3	2
Bin 2	5, 7	2
Bin 3	9, 12	2

| Bin 4 | 15, 18 | 2 |
| Bin 5 | 22, 25 | 2 |

Now notice the interval widths.

- First bin: 2–3
- Second bin: 5–7
- Third bin: 9–12
- Fourth bin: 15–18
- Fifth bin: 22–25

The widths are different.

But each bin contains two observations.

That is Equal Frequency Binning.

5.3 Understanding Percentiles and Quantiles

To understand Quantile Binning, you need to understand percentiles.

A percentile tells us the value below which a certain percentage of observations falls.

For example:

- 10th percentile: Approximately 10% of observations lie at or below this value.
- 20th percentile: Approximately 20% lie at or below this value.
- 50th percentile: Approximately 50% lie at or below this value.
- 90th percentile: Approximately 90% lie at or below this value.

Suppose the 10th percentile of Age is 16.

It means approximately 10% of the observations have ages at or below 16.

Suppose the 20th percentile is 20.

It means approximately 20% of observations have ages at or below 20.

If we want 10 bins, we can use these percentile boundaries:

$$
10\%,20\%,30\%,\ldots,100\%
$$

This creates ten groups with approximately equal numbers of observations.

5.4 How does Quantile Binning work?

The process is:

1. Take the numerical feature.
2. Decide the number of bins.
3. Sort the observations.
4. Calculate the required quantile boundaries.
5. Create intervals using those boundaries.
6. Assign each observation to its corresponding interval.

The important observation is that the width of the intervals changes depending on the distribution of the data.

If many observations are concentrated in a small range, the bin width in that region becomes smaller.

If observations are spread over a large range, the corresponding bin width becomes larger.

5.5 Why is Quantile Binning generally preferred?

The instructor highlights two important advantages.

Advantage 1: Handles outliers

Suppose your dataset contains:

\[
[10,12,14,16,18,20,22,24,26,500]
\]

The value 500 is extremely large compared with the other values.

Quantile Binning can place it in the final bin instead of allowing it to dominate the width of every interval.

It does not delete the outlier, but it can reduce its influence on the bin boundaries.

Advantage 2: Makes the frequency distribution more uniform

Every bin contains approximately the same number of observations.

This is especially useful when the original feature is highly skewed.

The instructor points out that Quantile Binning is generally used more often than Equal Width Binning because it can handle skewed distributions and produce more balanced bins.

5.6 Equal Width vs Equal Frequency

This comparison is extremely important for interviews and practical work.

Property	Equal Width	Equal Frequency
Other name	Uniform Binning	Quantile Binning
Bin width	Equal	May differ

Observations per bin	May differ	Approximately equal
Uses	Min/max range	Quantile boundaries
Outlier handling	Can be affected by extreme range	Usually more robust
Skewed data	May create empty or sparse bins	Often handles it better

Remember this simple distinction:

- Equal Width = Equal interval width.
- Equal Frequency = Equal number of observations.

PART 5 — K-MEANS BINNING

6. What is K-Means Binning?

The third unsupervised technique covered in the lecture is K-Means Binning.

To understand it, you first need to understand the basic idea of the K-Means clustering algorithm.

K-Means is a clustering algorithm.

Its purpose is to divide observations into K clusters based on their similarity.

In K-Means Binning, we use the same clustering concept to divide numerical values into groups.

The important difference is that we use the resulting clusters as bins.

6.1 Understanding K-Means with a numerical example

Suppose we have the following observations:

$$
X=[2,3,4,5,6,20,21,22,23,24,50,51,52,53]
$$

Look carefully at the data.

We can observe three natural groups:

Original numerical observations

{#each [2,3,4,5,6] as n}
[{n}]
{/each}
{#each [20,21,22,23,24] as n}
[{n}]
{/each}
{#each [50,51,52,53] as n}
[{n}]

{/each}

Three visibly separated groups of observations.

If we choose:

$$
K=3
$$

K-Means will attempt to create three clusters.

The clusters could be:

Cluster	Observations
Cluster 1	2, 3, 4, 5, 6
Cluster 2	20, 21, 22, 23, 24
Cluster 3	50, 51, 52, 53

These three clusters can now be treated as three bins.

6.2 How does K-Means work internally?

This is important because the instructor explains the internal working of K-Means in the lecture.

Let's understand it step by step.

Step 1: Decide the number of clusters

Suppose:

$$
K=3
$$

We want three clusters.

Step 2: Initialize centroids

A centroid is the central representative point of a cluster.

K-Means initially selects centroid positions.

For example:

$$
C_1=4
$$

$$
C_2=22
$$

$$
C_3=51
$$

These are illustrative initial centroids.

Step 3: Calculate distances

Now calculate the distance between each observation and every centroid.

For a one-dimensional feature, the distance is:

$$
d=|X-C|
$$

For example, consider:

$$
X=5
$$

The distances are:

$$
d_1=|5-4|=1
$$

$$
d_2=|5-22|=17
$$

$$
d_3=|5-51|=46
$$

The nearest centroid is C_1.

Therefore, the observation 5 belongs to Cluster 1.

We repeat this process for all observations.

Step 4: Recalculate the centroids

After assigning observations to clusters, calculate the mean of each cluster.

The centroid formula is:

$$
C_j=\frac{1}{N_j}\sum_{i=1}^{N_j}X_i
$$

Where:

- C_j = Centroid of cluster j.
- N_j = Number of observations in that cluster.
- X_i = Observations belonging to that cluster.

For example:

Suppose one cluster contains:

$$
[2,3,4,5,6]
$$

Its new centroid becomes:

$$
C_1=\frac{2+3+4+5+6}{5}
$$

$$
C_1=4
$$

The centroid is the average of all observations belonging to that cluster.

Step 5: Repeat the process

Now:

1. Calculate distances again.
2. Assign observations to their nearest centroid.
3. Recalculate the centroids.
4. Repeat until the centroids stop changing significantly or the algorithm reaches its stopping condition.

This is how K-Means forms clusters.

6.3 How does this become Binning?

Now connect K-Means to Discretization.

Once K-Means has created clusters:

- Cluster 1 becomes Bin 1.
- Cluster 2 becomes Bin 2.
- Cluster 3 becomes Bin 3.

Each observation receives the label of its assigned cluster.

Therefore, K-Means Binning converts numerical observations into cluster-based categories.

6.4 When should we use K-Means Binning?

The instructor specifically discusses using it when the data naturally appears in separate clusters.

For example, consider:

\[
[2,3,4,5,6,20,21,22,23,24,50,51,52,53]
\]

There are clear gaps between the groups.

In this situation, K-Means can identify the natural groups in the data.

It is particularly useful when:

- The data has distinct clusters.
- There are gaps between groups.
- Equal Width Binning would create poorly populated intervals.
- Quantile Binning would divide natural clusters unnecessarily.

However, K-Means Binning has some limitations:

- You need to select K.
- Results may depend on initialization.
- It can be affected by outliers.
- It may produce different boundaries depending on the data.

6.5 Comparing the three unsupervised methods

Feature	Equal Width	Equal Frequency	K-Means
Main idea	Equal interval width	Equal observations	Similarity-based clusters
Number of bins	User decides	User decides	User decides K
Bin width	Equal	Unequal	Determined by clusters
Frequency	Unequal	Approximately equal	Unequal
Best suited for	Simple numerical ranges	Skewed data	Naturally clustered data

---

PART 6 — IMPLEMENTATION OF BINNING IN SCIKIT-LEARN

7. KBinsDiscretizer

Scikit-learn provides a class called:

`KBinsDiscretizer`

It is available inside:

from sklearn.preprocessing import KBinsDiscretizer

This class provides an easy way to perform discretization.

The instructor explains three important parameters.

7.1 Parameter 1: n_bins

This determines the number of bins.

For example:

KBinsDiscretizer(n_bins=5)

This means that we want five bins.

You can also provide a different number of bins for each feature using an array-like value.

For example:

n_bins=[3, 5, 4]

This requests:

- 3 bins for the first feature.
- 5 bins for the second feature.
- 4 bins for the third feature.

7.2 Parameter 2: strategy

This determines which binning technique you want to use.

The instructor discusses three options.

Strategy	Meaning
`uniform`	Equal Width Binning
`quantile`	Equal Frequency Binning
`kmeans`	K-Means Binning

Examples:

```
KBinsDiscretizer(
n_bins=5,
strategy='uniform'
)
KBinsDiscretizer(
n_bins=5,
strategy='quantile'
)
KBinsDiscretizer(
n_bins=5,
strategy='kmeans'
)
```

7.3 Parameter 3: encode

This parameter determines how the resulting bins are represented.

The instructor explains two options:

1. Ordinal Encoding

```
encode='ordinal'
```

Every bin is represented using an integer.

For example:

Original value	Bin	Encoded value
5	Bin 1	0
15	Bin 2	1
25	Bin 3	2

This produces a single numerical column.

2. One-Hot Encoding

```
encode='onehot'
```

Every bin gets its own binary column.

For example:

Original value	Bin 1	Bin 2	Bin 3
5	1	0	0
15	0	1	0
25	0	0	1

This creates a separate column for each bin.

The instructor notes that Ordinal Encoding is often a convenient choice for binning because the bins have an

inherent numerical order, although the choice depends on the downstream model and representation.

7.4 Complete basic implementation

```
from sklearn.preprocessing import KBinsDiscretizer

kbd = KBinsDiscretizer(
n_bins=5,
strategy='quantile',
encode='ordinal'
)

X_binned = kbd.fit_transform(X)
```

Let's understand this code.

- `n_bins=5`: Create five bins.
- `strategy='quantile'`: Use Equal Frequency Binning.
- `encode='ordinal'`: Represent bins using integer labels.
- `fit_transform()`: Learn the bin boundaries and transform the data.

7.5 Inspecting the bin boundaries

After fitting:

```
kbd.bin_edges_
```

This returns the learned boundaries of the bins for each feature.

You can also inspect the number of bins actually created:

```
kbd.n_bins_
```

The number can be smaller than requested if there are insufficient distinct values or repeated quantile boundaries.

7.6 Understanding the output

Suppose:

```
X = [[10], [20], [30], [40], [50], [60]]
```

You apply:

```
kbd = KBinsDiscretizer(
n_bins=3,
strategy='uniform',
encode='ordinal'
)

X_new = kbd.fit_transform(X)
```

The output will contain integer bin labels.

The exact assignment depends on the learned boundaries and the boundary convention.

One important thing:

With ordinal encoding, Scikit-learn generally starts the bin labels from zero.

So the bins are represented as:

$$
0,1,2,\ldots,K-1
$$

PART 7 — PRACTICAL EXAMPLE USING TITANIC DATASET

8. Understanding the experiment

Now we move to the practical implementation demonstrated in the lecture.

The instructor uses the Titanic dataset.

The objective is to demonstrate whether binning can improve the performance of a machine learning model.

The instructor selects numerical columns such as:

- Age
- Fare

The target variable is:

- Survived

This is a classification problem because the target has two possible outcomes:

$$
0=\text{Did not survive}
$$

$$
1=\text{Survived}
$$

The instructor also handles missing values before applying binning.

8.1 Importing libraries

```
import numpy as np
import pandas as pd

from sklearn.model_selection import (
train_test_split,
cross_val_score
)

from sklearn.preprocessing import KBinsDiscretizer
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
```

8.2 Selecting features

The instructor focuses on:

```
X = df[['Age', 'Fare']]

y = df['Survived']
```

We are using only two input features to make the demonstration easier.

8.3 Handling missing values

The Age column contains missing values.

The instructor removes rows containing missing values for the selected columns.

```
df = df.dropna(subset=['Age', 'Fare'])
```

This is what the lecture demonstrates.

In a real project, you could also use an imputer rather than deleting observations.

8.4 Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```

Now we have separate training and testing datasets.

8.5 Building the baseline model without binning

The instructor first builds a Decision Tree Classifier using the original numerical features.

```
clf = DecisionTreeClassifier()

clf.fit(X_train, y_train)
```

y_pred = clf.predict(X_test)

Then evaluates the model using accuracy and cross-validation.

from sklearn.metrics import accuracy_score

print(accuracy_score(y_test, y_pred))

The lecture reports a baseline accuracy of approximately 62.9% in its experiment.

The exact result can vary depending on the dataset and split.

CHATGPT (Message 12)

8.6 Applying Quantile Binning to Age and Fare

Now we will apply binning to the input features.

The instructor creates two separate discretizer objects.

from sklearn.preprocessing import KBinsDiscretizer

```
kb1 = KBinsDiscretizer(
n_bins=10,
encode='ordinal',
strategy='quantile'
)

kb2 = KBinsDiscretizer(
n_bins=10,
encode='ordinal',
strategy='quantile'
)
```

Why create two objects?

Because we may want to experiment with different settings for Age and Fare.

For example:

- `kb1` can be used for Age.
- `kb2` can be used for Fare.

We can also reuse one object when both columns need the same strategy and number of bins.

8.7 Applying transformations using ColumnTransformer

The instructor uses `ColumnTransformer` to apply the transformations to the selected columns.

from sklearn.compose import ColumnTransformer

```
trf = ColumnTransformer([
('first', kb1, [0]),
('second', kb2, [1])
], remainder='passthrough')
```

Here:

- `first`: Applies the first transformer to column 0 (Age).
- `second`: Applies the second transformer to column 1 (Fare).
- `remainder='passthrough'`: Keeps any other columns unchanged.

Since we selected only Age and Fare, there are no additional columns in this particular example.

Now fit the transformer:

```
X_train_trf = trf.fit_transform(X_train)
```

```
X_test_trf = trf.transform(X_test)
```

Important: The bin boundaries are learned from the training data and reused for the testing data.

8.8 Training the Decision Tree after Binning

Now train the classifier on the transformed features.

```
clf = DecisionTreeClassifier()
```

```
clf.fit(X_train_trf, y_train)
```

```
y_pred = clf.predict(X_test_trf)
```

```
print(accuracy_score(y_test, y_pred))
```

The instructor observes that the result changes after applying binning.

The experiment is repeated with different strategies.

The important learning is that binning can change the way a Decision Tree sees numerical features.

8.9 Comparing Quantile and Uniform Binning

Now change the strategy:

```
strategy='uniform'
```

The instructor observes that the resulting distribution is different from Quantile Binning.

With Quantile Binning:

- The number of observations in each bin is approximately equal.
- Bin widths may differ.

With Uniform Binning:

- Bin widths are equal.
- Number of observations in each bin may differ.

The lecture reports an accuracy of approximately 63.01% for its Quantile Binning experiment.

The instructor also notes that the performance improvement is small and that binning does not necessarily improve every feature or every experiment.

Important learning: Never apply binning just because it is available. Compare the model before and after transformation.

8.10 Understanding the effect on Age and Fare

The instructor plots the distributions before and after transformation.

The purpose is to observe how the original numerical features are converted into bins.

For example, an Age distribution might originally contain many different numerical values.

After Quantile Binning with ten bins, it contains up to ten categories.

Similarly, Fare gets divided into its own bins.

You can inspect the actual boundaries:

```
trf.named_transformers_['first'].bin_edges_
```

And for Fare:

```
trf.named_transformers_['second'].bin_edges_
```

This lets you understand exactly which ranges of Age and Fare belong to which bins.

PART 8 — CUSTOM BINNING

9. What is Custom Binning?

Until now, we have used algorithms to create bin boundaries.

But what if we already know the correct categories from our domain knowledge?

For example, suppose you are building an application that categorizes people based on their age.

You may decide:

- 0–18: Minor
- 18–60: Adult

- 60 and above: Senior

You do not need Equal Width, Equal Frequency or K-Means to create these categories.

You can define the boundaries yourself.

This is called **Custom Binning** or Domain-Based Binning.

The instructor explains that custom binning is useful when you have domain knowledge about the feature.

9.1 Example: Age Categories

Suppose:

$$
Age=[12,17,25,35,45,65,75]
$$

We decide the categories:

Age range	Category
0–18	Minor
18–60	Adult
60+	Senior

The resulting data becomes:

Age	Category
12	Minor
17	Minor
25	Adult
35	Adult
45	Adult
65	Senior
75	Senior

Here, the bin boundaries are selected using our knowledge rather than an automatic algorithm.

9.2 Why use Custom Binning?

Imagine you are building a financial application.

You want to categorize annual income:

- Less than ■3 lakh: Low income
- ■3 lakh to ■10 lakh: Middle income

- More than ■10 lakh: High income

These boundaries may be chosen based on the application's purpose, business rules or relevant domain knowledge.

An automatic algorithm may create completely different boundaries that are not meaningful for your application.

Custom Binning gives you control.

9.3 Implementing Custom Binning using Pandas

One simple way is to use `pd.cut()`.

```
import pandas as pd

age = [12, 17, 25, 35, 45, 65, 75]

bins = [0, 18, 60, 100]

labels = ['Minor', 'Adult', 'Senior']

age_categories = pd.cut(
age,
bins=bins,
labels=labels,
include_lowest=True
)

print(age_categories)
```

Here:

- `bins` contains the interval boundaries.
- `labels` contains the names of the categories.
- `include_lowest=True` includes the lowest boundary.

Remember that Pandas uses a particular interval convention. You should always check the interval boundaries when defining categories.

9.4 Can we use Custom Binning in Scikit-learn?

The instructor explains that the `KBinsDiscretizer` class does not provide a direct custom-boundary strategy.

Therefore, if you want completely customized boundaries, you can define your own transformation logic.

For example:

```
def categorize_age(age):
```

if age 6,00,000
\end{cases}
\]

This creates a binary feature.

Note: This is a simplified illustration from the lecture, not a statement of actual Indian tax rules.

10.1 Understanding Binarization with an example

Suppose the annual income values are:

\[
X=[2,4,5,7,9,12]
\]

Assume our threshold is 6.

Then:

Income (in lakh)	Binary output
2	0
4	0
5	0
7	1
9	1
12	1

All values at or below the threshold become 0.

All values above the threshold become 1.

This is Binarization.

10.2 Understanding the threshold

The threshold determines where the conversion happens.

Suppose:

\[
Threshold=6
\]

Then:

- Values at or below 6 become 0.
- Values above 6 become 1.

If we change the threshold to 10, the output will change.

Therefore, selecting the correct threshold is important.

10.3 Real-world application: Image Processing

The instructor also gives an example of image processing.

A grayscale image contains pixel values.

For a standard 8-bit grayscale image:

$$
Pixel\ Value\in[0,255]
$$

Here:

- 0 represents black.
- 255 represents white.
- Intermediate values represent shades of gray.

Suppose we want to convert a grayscale image into a black-and-white image.

We choose a threshold:

$$
Threshold=127
$$

Then:

$$
Pixel'=
\begin{cases}
0, & Pixel\leq127\\
1, & Pixel>127
\end{cases}
$$

The result is a binary image.

Pixels below the threshold become black.

Pixels above the threshold become white.

This is a common application of Binarization.

PART 10 — IMPLEMENTING BINARIZATION IN SCIKIT-LEARN

11. Binarizer Class

Scikit-learn provides a class called:

`Binarizer`

Import it using:

from sklearn.preprocessing import Binarizer

It has two important parameters in the lecture.

1. threshold

Determines the value at which the binary conversion happens.

2. copy

Determines whether the original input should be copied or modified in place when possible.

Let's understand the implementation.

11.1 Creating a Binarizer object

```
from sklearn.preprocessing import Binarizer

bn = Binarizer(
threshold=5,
copy=True
)
```

This creates a Binarizer with threshold 5.

11.2 Applying the transformation

```
X = [[1], [3], [5], [7], [9]]

X_new = bn.fit_transform(X)

print(X_new)
```

Output:

```
[[0],
[0],
[0],
```

[1],
[1]]

Why?

Because Scikit-learn's Binarizer uses:

$$
X>threshold \Rightarrow 1
$$

$$
X\leq threshold \Rightarrow 0
$$

Therefore:

- 1 becomes 0.
- 3 becomes 0.
- 5 becomes 0.
- 7 becomes 1.
- 9 becomes 1.

Notice that a value exactly equal to the threshold becomes zero.

This is an important detail.

11.3 Understanding copy=True and copy=False

```
bn = Binarizer(
threshold=5,
copy=True
)
```

With `copy=True`, Scikit-learn tries to preserve the original input and produce a transformed copy.

With `copy=False`, it may perform the transformation in place when possible.

Be careful when using `copy=False`, because the original data may be modified.

PART 11 — PRACTICAL BINARIZATION USING TITANIC DATASET

12. Understanding the instructor's experiment

In the final practical example, the instructor again uses the Titanic dataset.

This time, the objective is to convert multiple numerical features into a single binary feature.

The instructor uses:

- SibSp
- Parch

Let's understand these two columns.

SibSp: Number of siblings or spouses aboard the Titanic.

Parch: Number of parents or children aboard the Titanic.

Suppose a passenger has:

SibSp = 1
Parch = 2

This means the passenger is travelling with family members.

But suppose:

SibSp = 0
Parch = 0

This means the passenger has no siblings, spouses, parents or children recorded as travelling with them.

The instructor wants to create a new feature:

`Family`

12.1 Creating the Family feature

The instructor combines the two columns.

df['Family'] = df['SibSp'] + df['Parch']

Now the feature represents the number of family members travelling with the passenger, excluding the passenger themselves.

For example:

SibSp	Parch	Family
0	0	0
1	0	1
0	2	2
1	2	3
2	3	5

12.2 Creating the binary feature

Now the instructor wants to identify whether a passenger is travelling alone.

The logic is:

\[
Family=0 \Rightarrow Travelling\ Alone=1
\]

\[
Family>0 \Rightarrow Travelling\ Alone=0
\]

Notice the meaning:

- 1 means travelling alone.
- 0 means not travelling alone.

This is the reverse of some common binary conventions, so always check what 0 and 1 represent.

12.3 Implementing Binarizer

```
from sklearn.preprocessing import Binarizer

bn = Binarizer(
threshold=0,
copy=False
)
```

Now apply it to the Family column.

```
df['Family'] = bn.fit_transform(
df[['Family']]
)
```

This produces:

Original Family	Binarized Family
0	0
1	1
2	1
3	1
5	1

But remember, the Binarizer itself gives 0 for values at or below zero and 1 for values above zero.

Therefore, the resulting output actually means:

- 0 = Travelling alone.
- 1 = Travelling with family.

If you specifically want `1` to mean travelling alone, invert the result:

```
df['Travelling_Alone'] = (
1 - bn.fit_transform(df[['Family']])
)
```

This distinction is important when interpreting binary features.

12.4 Does Binarization improve model performance?

The instructor compares the model before and after binarization.

The improvement is not particularly significant in the demonstrated experiment.

The instructor also explains that this is an illustrative example rather than a strong claim that binarization will always improve performance.

The important learning is that we can create meaningful binary features using existing numerical columns.

CHATGPT (Message 13)

Lecture 32: Binning and Binarization

Machine Learning — Feature Engineering | Complete offline study notes

[Complete transcript covered]
[Lecture 32]

Topics covered in this lecture:

1. Discretization (Binning)
2. Equal Width Binning
3. Equal Frequency Binning (Quantile Binning)
4. K-Means Binning
5. Custom Binning
6. Binarization
7. Scikit-learn implementation
8. Practical implementation using the Titanic dataset

The objective of this chapter is to understand how we convert numerical features into categorical or binary features, why we do it, and how to implement it in Scikit-learn.

PART 1 — DISCRETIZATION (BINNING)

1. Why do we need to convert numerical data into categorical data?

Suppose you are working on a Machine Learning project.

You have a dataset containing information about different mobile applications.

One column contains the number of downloads.

Application	Downloads
App A	500
App B	5,000
App C	50,000
App D	500,000
App E	5,000,000
App F	50,000,000
App G	100,000,000

Notice something.

Some applications have very few downloads, while some applications have millions of downloads.

The numerical values have a very large range.

Sometimes, working with the exact numerical values is not the most useful representation for a particular problem.

Instead, we could create categories:

Downloads	Category
500	Low downloads
5,000	Low downloads
50,000	Low downloads
500,000	Medium downloads
5,000,000	High downloads
50,000,000	Very high downloads
100,000,000	Very high downloads

Now the original numerical feature has been converted into categories.

This is the basic idea behind **Discretization**.

The instructor uses this type of example to explain why converting numerical data into categorical data can sometimes provide a better representation for a particular problem.

2. What is Discretization?

[Important definition]

Discretization is the process of converting continuous numerical values into a limited number of discrete intervals or categories.

It is also called **Binning**.

Let's understand the definition.

Suppose we have:

$$
X=[2,5,8,12,15,18,22,25,28]
$$

These are numerical values.

We want to divide them into three intervals:

- Bin 1: 0–10
- Bin 2: 10–20
- Bin 3: 20–30

Now each observation belongs to one interval.

Original value	Assigned bin
2	0–10
5	0–10
8	0–10
12	10–20
15	10–20
18	10–20
22	20–30
25	20–30
28	20–30

Instead of working with the exact values, we now work with their corresponding intervals.

This is called Binning.

2.1 Understanding bins

A bin is simply an interval or group into which numerical values are placed.

For example:

$$
[0,10),[10,20),[20,30]
$$

These are three bins.

The first bin contains values from 0 up to, but not including, 10.

The second bin contains values from 10 up to, but not including, 20.

The third bin contains values from 20 to 30.

The exact boundary convention depends on the implementation.

2.2 How does binning work?

The process is straightforward:

1. Take a numerical feature.
2. Decide how many bins you want.
3. Create intervals.
4. Assign every observation to the appropriate interval.
5. Represent the interval using a category or numerical code.

You can think of it as converting a continuous range into a limited number of groups.

2.3 Why do we use Discretization?

The instructor discusses three important benefits.

1. Handling outliers

Suppose your age feature contains:

$$
[18,20,22,25,28,30,35,40,150]
$$

The value 150 is an extreme value.

If we create bins:

- 0–20
- 20–40
- 40–60
- 60–80
- 80–100
- 100–160

The extreme value gets assigned to the final bin.

The model works with its bin representation rather than treating 150 as a separate extreme numerical value.

This can reduce the influence of extreme values in some modelling approaches.

However, remember that binning does not remove an outlier. It only changes its representation.

2. Improving the spread of data

Sometimes numerical observations are concentrated in a small region, while a few observations are spread over a much larger range.

Depending on the binning method, we can create categories that distribute observations more evenly.

We will understand this properly when we study Quantile Binning.

3. Simplifying the representation

Instead of handling hundreds or thousands of different numerical values, we can work with a smaller number of categories.

For example:

- Age 18
- Age 19
- Age 20
- Age 21
- Age 22

Can be represented using:

- Young
- Adult
- Middle-aged
- Senior

The categories can be easier to interpret in certain applications.

PART 2 — TYPES OF BINNING

3. Classification of Discretization techniques

The instructor divides binning into two main categories.

Discretization techniques

Discretization / Binning

Unsupervised

1. Equal Width Binning

2. Equal Frequency Binning

3. K-Means Binning

Supervised

1. Decision Tree Binning

The instructor focuses on the three unsupervised techniques and custom binning.

Decision Tree Binning is mentioned but not explained in detail in this lecture.

Before going further, understand the difference.

Unsupervised Binning: The binning process does not use the target variable.

Supervised Binning: The target variable is used when determining how the numerical feature should be divided.

For example, if you are predicting whether a customer will purchase a product, supervised binning can use the purchase outcome while deciding the bin boundaries.

PART 3 — EQUAL WIDTH BINNING

4. What is Equal Width Binning?

Equal Width Binning is a discretization technique in which every bin has the same numerical width.

The instructor also calls it:

- Uniform Binning
- Equal Width Discretization

The number of bins is decided by us.

Let's understand this with an example.

Suppose we have:

\[

$$
X=[2,5,8,12,15,18,22,25,28]
$$

The minimum value is:

$$
X_{\min}=2
$$

The maximum value is:

$$
X_{\max}=28
$$

Suppose we want three bins.

The formula for bin width is:

$$
\text{Bin Width}=
\frac{\text{Maximum}-\text{Minimum}}
{\text{Number of bins}}
$$

Substituting the values:

$$
=\frac{28-2}{3}
$$

$$
=\frac{26}{3}
$$

$$
=8.67
$$

Therefore, each bin will have approximately 8.67 units of width.

4.1 Creating the intervals

Starting from the minimum value, we create three intervals.

Bin	Interval
Bin 1	2–10.67
Bin 2	10.67–19.33
Bin 3	19.33–28

Notice something important:

All three intervals have approximately the same width.

That is why it is called Equal Width Binning.

4.2 Assigning observations

Our original data:

\[
[2,5,8,12,15,18,22,25,28]
\]

After assigning the values:

Value	Bin
2	1
5	1
8	1
12	2
15	2
18	2
22	3
25	3
28	3

The distribution of observations across bins is:

Bin	Number of observations
Bin 1	3
Bin 2	3
Bin 3	3

In this example, every bin happens to contain three observations.

But that is not guaranteed by Equal Width Binning.

If your data is highly skewed, one bin might contain 100 observations while another contains only 2.

This is a very important distinction.

4.3 Visual understanding

Here, the intervals have equal width, but the number of observations in each interval can differ.

4.4 Advantages of Equal Width Binning

1. Simple to understand

The formula is straightforward.

2. Easy to implement

You only need the minimum, maximum and number of bins.

3. Helps limit extreme numerical values

Very large values can be grouped into the final interval.

4.5 Disadvantages of Equal Width Binning

The biggest limitation is that it does not guarantee equal numbers of observations in each bin.

For example:

Suppose 900 observations are between 0 and 20, while only 100 observations are between 80 and 100.

If we divide the range into five equal-width bins, the first bins may contain most of the observations.

The last bins may contain very few observations.

Therefore, Equal Width Binning does not necessarily make the distribution of observations across bins uniform.

PART 4 — EQUAL FREQUENCY BINNING (QUANTILE BINNING)

5. What is Equal Frequency Binning?

[Very important]

Equal Frequency Binning is another method of discretization.

It is also called:

- Quantile Binning
- Equal Frequency Discretization

In Equal Width Binning, every bin has the same width.

But in Equal Frequency Binning, every bin contains approximately the same number of observations.

This is the biggest difference.

Let's understand it properly.

5.1 Understanding the concept

Suppose you have 100 students and their marks.

You want to divide them into five bins.

In Equal Frequency Binning, each bin should contain approximately:

$$
\frac{100}{5}=20
$$

students.

Therefore:

- Bin 1 contains 20 students.
- Bin 2 contains 20 students.
- Bin 3 contains 20 students.
- Bin 4 contains 20 students.
- Bin 5 contains 20 students.

Notice that we are not trying to make the numerical width of every interval equal.

We are trying to make the number of observations equal.

5.2 A numerical example

Suppose we have the following sorted data:

$$
X=[2,3,5,7,9,12,15,18,22,25]
$$

There are 10 observations.

Suppose we want five bins.

Each bin should contain:

$$
\frac{10}{5}=2
$$

observations.

We can create the following bins:

Bin	Values	Frequency
Bin 1	2, 3	2
Bin 2	5, 7	2
Bin 3	9, 12	2
Bin 4	15, 18	2
Bin 5	22, 25	2

Now notice the interval widths.

- First bin: 2–3
- Second bin: 5–7
- Third bin: 9–12
- Fourth bin: 15–18
- Fifth bin: 22–25

The widths are different.

But each bin contains two observations.

That is Equal Frequency Binning.

5.3 Understanding Percentiles and Quantiles

To understand Quantile Binning, you need to understand percentiles.

A percentile tells us the value below which a certain percentage of observations falls.

For example:

- 10th percentile: Approximately 10% of observations lie at or below this value.
- 20th percentile: Approximately 20% lie at or below this value.
- 50th percentile: Approximately 50% lie at or below this value.
- 90th percentile: Approximately 90% lie at or below this value.

Suppose the 10th percentile of Age is 16.

It means approximately 10% of the observations have ages at or below 16.

Suppose the 20th percentile is 20.

It means approximately 20% of observations have ages at or below 20.

If we want 10 bins, we can use these percentile boundaries:

$$
10\%,20\%,30\%,\ldots,100\%
$$

This creates ten groups with approximately equal numbers of observations.

5.4 How does Quantile Binning work?

The process is:

1. Take the numerical feature.
2. Decide the number of bins.
3. Sort the observations.
4. Calculate the required quantile boundaries.
5. Create intervals using those boundaries.
6. Assign each observation to its corresponding interval.

The important observation is that the width of the intervals changes depending on the distribution of the data.

If many observations are concentrated in a small range, the bin width in that region becomes smaller.

If observations are spread over a large range, the corresponding bin width becomes larger.

5.5 Why is Quantile Binning generally preferred?

The instructor highlights two important advantages.

Advantage 1: Handles outliers

Suppose your dataset contains:

\[
[10,12,14,16,18,20,22,24,26,500]
\]

The value 500 is extremely large compared with the other values.

Quantile Binning can place it in the final bin instead of allowing it to dominate the width of every interval.

It does not delete the outlier, but it can reduce its influence on the bin boundaries.

Advantage 2: Makes the frequency distribution more uniform

Every bin contains approximately the same number of observations.

This is especially useful when the original feature is highly skewed.

The instructor points out that Quantile Binning is generally used more often than Equal Width Binning because it can handle skewed distributions and produce more balanced bins.

5.6 Equal Width vs Equal Frequency

This comparison is extremely important for interviews and practical work.

Property	Equal Width	Equal Frequency
Other name	Uniform Binning	Quantile Binning
Bin width	Equal	May differ
Observations per bin	May differ	Approximately equal
Uses	Min/max range	Quantile boundaries
Outlier handling	Can be affected by extreme range	Usually more robust
Skewed data	May create empty or sparse bins	Often handles it better

Remember this simple distinction:

- Equal Width = Equal interval width.
- Equal Frequency = Equal number of observations.

PART 5 — K-MEANS BINNING

6. What is K-Means Binning?

The third unsupervised technique covered in the lecture is K-Means Binning.

To understand it, you first need to understand the basic idea of the K-Means clustering algorithm.

K-Means is a clustering algorithm.

Its purpose is to divide observations into K clusters based on their similarity.

In K-Means Binning, we use the same clustering concept to divide numerical values into groups.

The important difference is that we use the resulting clusters as bins.

6.1 Understanding K-Means with a numerical example

Suppose we have the following observations:

$$
X=[2,3,4,5,6,20,21,22,23,24,50,51,52,53]
$$

Look carefully at the data.

We can observe three natural groups:

Original numerical observations

{#each [2,3,4,5,6] as n}
[{n}]

{/each}
{#each [20,21,22,23,24] as n}
[{n}]
{/each}
{#each [50,51,52,53] as n}
[{n}]
{/each}

Three visibly separated groups of observations.

If we choose:

$$
K=3
$$

K-Means will attempt to create three clusters.

The clusters could be:

Cluster	Observations
Cluster 1	2, 3, 4, 5, 6
Cluster 2	20, 21, 22, 23, 24
Cluster 3	50, 51, 52, 53

These three clusters can now be treated as three bins.

6.2 How does K-Means work internally?

This is important because the instructor explains the internal working of K-Means in the lecture.

Let's understand it step by step.

Step 1: Decide the number of clusters

Suppose:

$$
K=3
$$

We want three clusters.

Step 2: Initialize centroids

A centroid is the central representative point of a cluster.

K-Means initially selects centroid positions.

For example:

$$
C_1=4
$$

$$
C_2=22
$$

$$
C_3=51
$$

These are illustrative initial centroids.

Step 3: Calculate distances

Now calculate the distance between each observation and every centroid.

For a one-dimensional feature, the distance is:

$$
d=|X-C|
$$

For example, consider:

$$
X=5
$$

The distances are:

$$
d_1=|5-4|=1
$$

$$
d_2=|5-22|=17
$$

$$
d_3=|5-51|=46
$$

The nearest centroid is C_1.

Therefore, the observation 5 belongs to Cluster 1.

We repeat this process for all observations.

Step 4: Recalculate the centroids

After assigning observations to clusters, calculate the mean of each cluster.

The centroid formula is:

$$
C_j=\frac{1}{N_j}\sum_{i=1}^{N_j}X_i
$$

Where:

- C_j = Centroid of cluster j.
- N_j = Number of observations in that cluster.
- X_i = Observations belonging to that cluster.

For example:

Suppose one cluster contains:

$$
[2,3,4,5,6]
$$

Its new centroid becomes:

$$
C_1=\frac{2+3+4+5+6}{5}
$$

$$
C_1=4
$$

The centroid is the average of all observations belonging to that cluster.

Step 5: Repeat the process

Now:

1. Calculate distances again.
2. Assign observations to their nearest centroid.
3. Recalculate the centroids.
4. Repeat until the centroids stop changing significantly or the algorithm reaches its stopping condition.

This is how K-Means forms clusters.

6.3 How does this become Binning?

Now connect K-Means to Discretization.

Once K-Means has created clusters:

- Cluster 1 becomes Bin 1.
- Cluster 2 becomes Bin 2.
- Cluster 3 becomes Bin 3.

Each observation receives the label of its assigned cluster.

Therefore, K-Means Binning converts numerical observations into cluster-based categories.

6.4 When should we use K-Means Binning?

The instructor specifically discusses using it when the data naturally appears in separate clusters.

For example, consider:

\[
[2,3,4,5,6,20,21,22,23,24,50,51,52,53]
\]

There are clear gaps between the groups.

In this situation, K-Means can identify the natural groups in the data.

It is particularly useful when:

- The data has distinct clusters.
- There are gaps between groups.
- Equal Width Binning would create poorly populated intervals.
- Quantile Binning would divide natural clusters unnecessarily.

However, K-Means Binning has some limitations:

- You need to select K.
- Results may depend on initialization.
- It can be affected by outliers.
- It may produce different boundaries depending on the data.

6.5 Comparing the three unsupervised methods

Feature	Equal Width	Equal Frequency	K-Means
Main idea	Equal interval width	Equal observations	Similarity-based clusters
Number of bins	User decides	User decides	User decides K
Bin width	Equal	Unequal	Determined by clusters

| Frequency | Unequal | Approximately equal | Unequal |
| Best suited for | Simple numerical ranges | Skewed data | Naturally clustered data |

PART 6 — IMPLEMENTATION OF BINNING IN SCIKIT-LEARN

7. KBinsDiscretizer

Scikit-learn provides a class called:

`KBinsDiscretizer`

It is available inside:

from sklearn.preprocessing import KBinsDiscretizer

This class provides an easy way to perform discretization.

The instructor explains three important parameters.

7.1 Parameter 1: n_bins

This determines the number of bins.

For example:

KBinsDiscretizer(n_bins=5)

This means that we want five bins.

You can also provide a different number of bins for each feature using an array-like value.

For example:

n_bins=[3, 5, 4]

This requests:

- 3 bins for the first feature.
- 5 bins for the second feature.
- 4 bins for the third feature.

7.2 Parameter 2: strategy

This determines which binning technique you want to use.

The instructor discusses three options.

Strategy	Meaning

`uniform`	Equal Width Binning
`quantile`	Equal Frequency Binning
`kmeans`	K-Means Binning

Examples:

```
KBinsDiscretizer(
n_bins=5,
strategy='uniform'
)
```

```
KBinsDiscretizer(
n_bins=5,
strategy='quantile'
)
```

```
KBinsDiscretizer(
n_bins=5,
strategy='kmeans'
)
```

7.3 Parameter 3: encode

This parameter determines how the resulting bins are represented.

The instructor explains two options:

1. Ordinal Encoding

```
encode='ordinal'
```

Every bin is represented using an integer.

For example:

Original value	Bin	Encoded value
5	Bin 1	0
15	Bin 2	1
25	Bin 3	2

This produces a single numerical column.

2. One-Hot Encoding

```
encode='onehot'
```

Every bin gets its own binary column.

For example:

Original value	Bin 1	Bin 2	Bin 3

5	1	0	0
15	0	1	0
25	0	0	1

This creates a separate column for each bin.

The instructor notes that Ordinal Encoding is often a convenient choice for binning because the bins have an inherent numerical order, although the choice depends on the downstream model and representation.

7.4 Complete basic implementation

```
from sklearn.preprocessing import KBinsDiscretizer

kbd = KBinsDiscretizer(
n_bins=5,
strategy='quantile',
encode='ordinal'
)

X_binned = kbd.fit_transform(X)
```

Let's understand this code.

- `n_bins=5`: Create five bins.
- `strategy='quantile'`: Use Equal Frequency Binning.
- `encode='ordinal'`: Represent bins using integer labels.
- `fit_transform()`: Learn the bin boundaries and transform the data.

7.5 Inspecting the bin boundaries

After fitting:

```
kbd.bin_edges_
```

This returns the learned boundaries of the bins for each feature.

You can also inspect the number of bins actually created:

```
kbd.n_bins_
```

The number can be smaller than requested if there are insufficient distinct values or repeated quantile boundaries.

7.6 Understanding the output

Suppose:

```
X = [[10], [20], [30], [40], [50], [60]]
```

You apply:

```
kbd = KBinsDiscretizer(
n_bins=3,
strategy='uniform',
encode='ordinal'
)

X_new = kbd.fit_transform(X)
```

The output will contain integer bin labels.

The exact assignment depends on the learned boundaries and the boundary convention.

One important thing:

With ordinal encoding, Scikit-learn generally starts the bin labels from zero.

So the bins are represented as:

$$
0,1,2,\ldots,K-1
$$

PART 7 — PRACTICAL EXAMPLE USING TITANIC DATASET

8. Understanding the experiment

Now we move to the practical implementation demonstrated in the lecture.

The instructor uses the Titanic dataset.

The objective is to demonstrate whether binning can improve the performance of a machine learning model.

The instructor selects numerical columns such as:

- Age
- Fare

The target variable is:

- Survived

This is a classification problem because the target has two possible outcomes:

$$
0=\text{Did not survive}
$$

$$
1=\text{Survived}
$$

\]

The instructor also handles missing values before applying binning.

8.1 Importing libraries

```
import numpy as np
import pandas as pd

from sklearn.model_selection import (
train_test_split,
cross_val_score
)

from sklearn.preprocessing import KBinsDiscretizer
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
```

8.2 Selecting features

The instructor focuses on:

```
X = df[['Age', 'Fare']]

y = df['Survived']
```

We are using only two input features to make the demonstration easier.

8.3 Handling missing values

The Age column contains missing values.

The instructor removes rows containing missing values for the selected columns.

```
df = df.dropna(subset=['Age', 'Fare'])
```

This is what the lecture demonstrates.

In a real project, you could also use an imputer rather than deleting observations.

8.4 Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```

Now we have separate training and testing datasets.

8.5 Building the baseline model without binning

The instructor first builds a Decision Tree Classifier using the original numerical features.

```
clf = DecisionTreeClassifier()

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
```

Then evaluates the model using accuracy and cross-validation.

```
from sklearn.metrics import accuracy_score

print(accuracy_score(y_test, y_pred))
```

The lecture reports a baseline accuracy of approximately 62.9% in its experiment.

The exact result can vary depending on the dataset and split.

8.6 Applying Quantile Binning to Age and Fare

Now we will apply binning to the input features.

The instructor creates two separate discretizer objects.

```
from sklearn.preprocessing import KBinsDiscretizer

kb1 = KBinsDiscretizer(
n_bins=10,
encode='ordinal',
strategy='quantile'
)

kb2 = KBinsDiscretizer(
n_bins=10,
encode='ordinal',
strategy='quantile'
)
```

Why create two objects?

Because we may want to experiment with different settings for Age and Fare.

For example:

- `kb1` can be used for Age.
- `kb2` can be used for Fare.

We can also reuse one object when both columns need the same strategy and number of bins.

8.7 Applying transformations using ColumnTransformer

The instructor uses `ColumnTransformer` to apply the transformations to the selected columns.

from sklearn.compose import ColumnTransformer

trf = ColumnTransformer([
('first', kb1, [0]),
('second', kb2, [1])
], remainder='passthrough')

Here:

- `first`: Applies the first transformer to column 0 (Age).
- `second`: Applies the second transformer to column 1 (Fare).
- `remainder='passthrough'`: Keeps any other columns unchanged.

Since we selected only Age and Fare, there are no additional columns in this particular example.

Now fit the transformer:

X_train_trf = trf.fit_transform(X_train)

X_test_trf = trf.transform(X_test)

Important: The bin boundaries are learned from the training data and reused for the testing data.

8.8 Training the Decision Tree after Binning

Now train the classifier on the transformed features.

clf = DecisionTreeClassifier()

clf.fit(X_train_trf, y_train)

y_pred = clf.predict(X_test_trf)

print(accuracy_score(y_test, y_pred))

The instructor observes that the result changes after applying binning.

The experiment is repeated with different strategies.

The important learning is that binning can change the way a Decision Tree sees numerical features.

8.9 Comparing Quantile and Uniform Binning

Now change the strategy:

strategy='uniform'

The instructor observes that the resulting distribution is different from Quantile Binning.

With Quantile Binning:

- The number of observations in each bin is approximately equal.
- Bin widths may differ.

With Uniform Binning:

- Bin widths are equal.
- Number of observations in each bin may differ.

The lecture reports an accuracy of approximately 63.01% for its Quantile Binning experiment.

The instructor also notes that the performance improvement is small and that binning does not necessarily improve every feature or every experiment.

Important learning: Never apply binning just because it is available. Compare the model before and after transformation.

8.10 Understanding the effect on Age and Fare

The instructor plots the distributions before and after transformation.

The purpose is to observe how the original numerical features are converted into bins.

For example, an Age distribution might originally contain many different numerical values.

After Quantile Binning with ten bins, it contains up to ten categories.

Similarly, Fare gets divided into its own bins.

You can inspect the actual boundaries:

```
trf.named_transformers_['first'].bin_edges_
```

And for Fare:

```
trf.named_transformers_['second'].bin_edges_
```

This lets you understand exactly which ranges of Age and Fare belong to which bins.

PART 8 — CUSTOM BINNING

9. What is Custom Binning?

Until now, we have used algorithms to create bin boundaries.

But what if we already know the correct categories from our domain knowledge?

For example, suppose you are building an application that categorizes people based on their age.

You may decide:

- 0–18: Minor
- 18–60: Adult
- 60 and above: Senior

You do not need Equal Width, Equal Frequency or K-Means to create these categories.

You can define the boundaries yourself.

This is called **Custom Binning** or Domain-Based Binning.

The instructor explains that custom binning is useful when you have domain knowledge about the feature.

9.1 Example: Age Categories

Suppose:

$$
Age=[12,17,25,35,45,65,75]
$$

We decide the categories:

Age range	Category
0–18	Minor
18–60	Adult
60+	Senior

The resulting data becomes:

Age	Category
12	Minor
17	Minor
25	Adult
35	Adult
45	Adult
65	Senior
75	Senior

Here, the bin boundaries are selected using our knowledge rather than an automatic algorithm.

9.2 Why use Custom Binning?

Imagine you are building a financial application.

You want to categorize annual income:

- Less than ■3 lakh: Low income
- ■3 lakh to ■10 lakh: Middle income
- More than ■10 lakh: High income

These boundaries may be chosen based on the application's purpose, business rules or relevant domain knowledge.

An automatic algorithm may create completely different boundaries that are not meaningful for your application.

Custom Binning gives you control.

9.3 Implementing Custom Binning using Pandas

One simple way is to use `pd.cut()`.

```
import pandas as pd

age = [12, 17, 25, 35, 45, 65, 75]

bins = [0, 18, 60, 100]

labels = ['Minor', 'Adult', 'Senior']

age_categories = pd.cut(
age,
bins=bins,
labels=labels,
include_lowest=True
)

print(age_categories)
```

Here:

- `bins` contains the interval boundaries.
- `labels` contains the names of the categories.
- `include_lowest=True` includes the lowest boundary.

Remember that Pandas uses a particular interval convention. You should always check the interval boundaries when defining categories.

9.4 Can we use Custom Binning in Scikit-learn?

The instructor explains that the `KBinsDiscretizer` class does not provide a direct custom-boundary strategy.

Therefore, if you want completely customized boundaries, you can define your own transformation logic.

For example:

def categorize_age(age):

if age 6,00,000
\end{cases}
\]

This creates a binary feature.

Note: This is a simplified illustration from the lecture, not a statement of actual Indian tax rules.

10.1 Understanding Binarization with an example

Suppose the annual income values are:

\[
X=[2,4,5,7,9,12]
\]

Assume our threshold is 6.

Then:

Income (in lakh)	Binary output
2	0
4	0
5	0
7	1
9	1
12	1

All values at or below the threshold become 0.

All values above the threshold become 1.

This is Binarization.

10.2 Understanding the threshold

The threshold determines where the conversion happens.

Suppose:

\[
Threshold=6
\]

Then:

- Values at or below 6 become 0.
- Values above 6 become 1.

If we change the threshold to 10, the output will change.

Therefore, selecting the correct threshold is important.

10.3 Real-world application: Image Processing

The instructor also gives an example of image processing.

A grayscale image contains pixel values.

For a standard 8-bit grayscale image:

$$
Pixel\ Value\in[0,255]
$$

Here:

- 0 represents black.
- 255 represents white.
- Intermediate values represent shades of gray.

Suppose we want to convert a grayscale image into a black-and-white image.

We choose a threshold:

$$
Threshold=127
$$

Then:

$$
Pixel'=
\begin{cases}
0, & Pixel\leq127\\
1, & Pixel>127
\end{cases}
$$

The result is a binary image.

Pixels below the threshold become black.

Pixels above the threshold become white.

This is a common application of Binarization.

PART 10 — IMPLEMENTING BINARIZATION IN SCIKIT-LEARN

11. Binarizer Class

Scikit-learn provides a class called:

`Binarizer`

Import it using:

from sklearn.preprocessing import Binarizer

It has two important parameters in the lecture.

1. threshold

Determines the value at which the binary conversion happens.

2. copy

Determines whether the original input should be copied or modified in place when possible.

Let's understand the implementation.

11.1 Creating a Binarizer object

```
from sklearn.preprocessing import Binarizer

bn = Binarizer(
threshold=5,
copy=True
)
```

This creates a Binarizer with threshold 5.

11.2 Applying the transformation

```
X = [[1], [3], [5], [7], [9]]

X_new = bn.fit_transform(X)

print(X_new)
```

Output:

[[0],
[0],
[0],
[1],
[1]]

Why?

Because Scikit-learn's Binarizer uses:

$$
X>threshold \Rightarrow 1
$$

$$
X\leq threshold \Rightarrow 0
$$

Therefore:

- 1 becomes 0.
- 3 becomes 0.
- 5 becomes 0.
- 7 becomes 1.
- 9 becomes 1.

Notice that a value exactly equal to the threshold becomes zero.

This is an important detail.

11.3 Understanding copy=True and copy=False

```
bn = Binarizer(
threshold=5,
copy=True
)
```

With `copy=True`, Scikit-learn tries to preserve the original input and produce a transformed copy.

With `copy=False`, it may perform the transformation in place when possible.

Be careful when using `copy=False`, because the original data may be modified.

PART 11 — PRACTICAL BINARIZATION USING TITANIC DATASET

12. Understanding the instructor's experiment

In the final practical example, the instructor again uses the Titanic dataset.

This time, the objective is to convert multiple numerical features into a single binary feature.

The instructor uses:

- SibSp
- Parch

Let's understand these two columns.

SibSp: Number of siblings or spouses aboard the Titanic.

Parch: Number of parents or children aboard the Titanic.

Suppose a passenger has:

SibSp = 1
Parch = 2

This means the passenger is travelling with family members.

But suppose:

SibSp = 0
Parch = 0

This means the passenger has no siblings, spouses, parents or children recorded as travelling with them.

The instructor wants to create a new feature:

`Family`

12.1 Creating the Family feature

The instructor combines the two columns.

df['Family'] = df['SibSp'] + df['Parch']

Now the feature represents the number of family members travelling with the passenger, excluding the passenger themselves.

For example:

SibSp	Parch	Family
0	0	0
1	0	1
0	2	2
1	2	3
2	3	5

12.2 Creating the binary feature

Now the instructor wants to identify whether a passenger is travelling alone.

The logic is:

\[
Family=0 \Rightarrow Travelling\ Alone=1
\]

\[
Family>0 \Rightarrow Travelling\ Alone=0
\]

Notice the meaning:

- 1 means travelling alone.
- 0 means not travelling alone.

This is the reverse of some common binary conventions, so always check what 0 and 1 represent.

12.3 Implementing Binarizer

```
from sklearn.preprocessing import Binarizer

bn = Binarizer(
threshold=0,
copy=False
)
```

Now apply it to the Family column.

```
df['Family'] = bn.fit_transform(
df[['Family']]
)
```

This produces:

Original Family	Binarized Family
0	0
1	1
2	1
3	1
5	1

But remember, the Binarizer itself gives 0 for values at or below zero and 1 for values above zero.

Therefore, the resulting output actually means:

- 0 = Travelling alone.
- 1 = Travelling with family.

If you specifically want `1` to mean travelling alone, invert the result:

```
df['Travelling_Alone'] = (
1 - bn.fit_transform(df[['Family']])
)
```

This distinction is important when interpreting binary features.

12.4 Does Binarization improve model performance?

The instructor compares the model before and after binarization.

The improvement is not particularly significant in the demonstrated experiment.

The instructor also explains that this is an illustrative example rather than a strong claim that binarization will always improve performance.

The important learning is that we can create meaningful binary features using existing numerical columns.

PART 12 — IMPORTANT COMPARISONS

13. Discretization vs Binarization

This is a common conceptual question.

Property	Discretization	Binarization
Meaning	Converts numerical values into intervals	Converts numerical values into two categories
Output categories	Multiple	Exactly two
Output example	0, 1, 2, 3	0, 1
Example	Age groups	Travelling alone or not
Scikit-learn	`KBinsDiscretizer`	`Binarizer`

Remember:

Binning can create multiple categories. Binarization creates only two categories.

14. Complete Binning Strategy Comparison

Technique	Main principle	Best situation
Equal Width	Equal interval widths	Data with manageable numerical ranges
Equal Frequency	Equal observations per bin	Skewed distributions
K-Means	Similarity-based clusters	Naturally separated groups
Custom Binning	Manually selected boundaries	Domain knowledge or business rules
Decision Tree Binning	Target-based splitting	Supervised feature discretization

One important clarification: Decision Tree Binning is mentioned in the lecture but its implementation is not covered in detail.

PART 13 — COMPLETE CODE REVISION

Here is a consolidated code reference for the entire lecture.

You can save this section separately for quick revision.

15. Equal Width Binning

```
from sklearn.preprocessing import KBinsDiscretizer

kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='uniform'
)

X_binned = kbd.fit_transform(X_train)
```

16. Equal Frequency Binning

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='quantile'
)

X_binned = kbd.fit_transform(X_train)
```

17. K-Means Binning

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='kmeans'
)

X_binned = kbd.fit_transform(X_train)
```

18. One-Hot Binning

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='onehot',
strategy='quantile'
)

X_binned = kbd.fit_transform(X_train)
```

This produces a sparse matrix by default.

If you specifically need a dense array:

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='onehot-dense',
strategy='quantile'
)
```

19. Custom Binning

```
import pandas as pd

df['Age_Category'] = pd.cut(
df['Age'],
bins=[0, 18, 60, 100],
labels=['Minor', 'Adult', 'Senior'],
include_lowest=True
)
```

20. Binarization

```
from sklearn.preprocessing import Binarizer

bn = Binarizer(
threshold=5,
copy=True
)

X_binary = bn.fit_transform(X)
```

21. Pipeline Implementation

A useful way to combine binning and a model is:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.tree import DecisionTreeClassifier

pipe = Pipeline([
('binning', KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='quantile'
)),
('model', DecisionTreeClassifier())
])

pipe.fit(X_train, y_train)

y_pred = pipe.predict(X_test)
```

This ensures the bin boundaries are learned from the training data and reused during prediction.

PART 14 — ADDITIONAL CONCEPTS TO UNDERSTAND

These points help connect the lecture to actual Machine Learning projects.

22. Does binning always improve model performance?

No.

Suppose you have a feature:

$$
Age=[21,22,23,24,25]
$$

If you convert all these values into one category called Young, you lose the difference between 21 and 25.

The original numerical information becomes less detailed.

This is called information loss.

Binning can sometimes improve performance, but it can also decrease performance.

It depends on:

- The dataset.
- The algorithm.
- The number of bins.
- The binning strategy.
- The relationship between the feature and the target.

23. Does binning remove outliers?

No.

This is an important distinction.

Suppose:

$$
X=[10,12,15,20,25,500]
$$

After binning, 500 might belong to the final bin.

But the original observation still exists.

We have only changed its representation.

Binning does not delete the outlier.

24. Does Ordinal Encoding introduce an ordering problem?

It can.

Suppose you convert Age into:

- Young = 0
- Adult = 1
- Senior = 2

These categories have a meaningful order.

But imagine you have:

- Red = 0
- Blue = 1
- Green = 2

There is no natural numerical order between these colors.

Assigning integer values can create an artificial relationship.

Therefore, One-Hot Encoding may be more appropriate for categories without a meaningful order.

For bins, ordinal encoding can be reasonable because the intervals have a natural order. However, the downstream algorithm matters.

25. Important implementation precautions

- Fit your binning transformer on training data only.
- Apply the learned boundaries to testing data using `transform()`.
- Check whether duplicate values create fewer distinct bins than requested.
- Choose the number of bins carefully.
- Compare performance before and after binning.
- Remember that ordinal bin labels are category identifiers, not necessarily the original numerical values.

PART 15 — QUICK REVISION NOTES

Lecture 32 — Cheat Sheet
1. Discretization

Converts continuous numerical values into discrete intervals.

2. Equal Width Binning

$$
Width=\frac{Max-Min}{K}
$$

- Equal interval width.
- Unequal frequency is possible.
- Scikit-learn strategy: `uniform`.

3. Equal Frequency Binning

- Approximately equal observations per bin.
- Unequal interval width.
- Based on quantiles.
- Scikit-learn strategy: `quantile`.

4. K-Means Binning

- Uses K-Means clustering.
- Assigns observations to clusters.
- Each cluster becomes a bin.
- Scikit-learn strategy: `kmeans`.

5. Custom Binning

- Manually selected intervals.
- Uses domain knowledge.
- Pandas: `pd.cut()`.

6. Binarization

Converts numerical data into 0 and 1.

- Uses a threshold.
- Scikit-learn: `Binarizer`.

7. Main Scikit-learn classes

KBinsDiscretizer()
Binarizer()

8. KBinsDiscretizer parameters

n_bins
strategy
encode

9. Binarizer parameters

threshold
copy

PART 16 — TEST YOUR UNDERSTANDING

Try these questions after completing the chapter.

{@body const questions = [{q:"What is Discretization?",a:["Converting numerical values into discrete intervals","Removing all outliers","Scaling data between 0 and 1","Converting text into numbers"],c:0,e:"Discretization divides continuous numerical values into a limited number of groups."},{q:"In Equal Width Binning, what remains equal?",a:["Number of observations","Interval width","Mean of each bin","Number of outliers"],c:1,e:"Each interval has the same numerical width, but frequencies may differ."},{q:"In Equal Frequency Binning, what remains approximately equal?",a:["Bin width","Number of observations per bin","Maximum value","Minimum value"],c:1,e:"Quantile Binning aims to put approximately equal numbers of observations in each bin."},{q:"Which strategy is used for K-Means Binning in Scikit-learn?",a:["uniform","quantile","kmeans","custom"],c:2,e:"Use strategy='kmeans' in KBinsDiscretizer."},{q:"Which class is used for discretization?",a:["Binarizer","KBinsDiscretizer","StandardScaler","SimpleImputer"],c:1,e:"KBinsDiscretizer performs numerical feature discretization."},{q:"What is the output of Binarization?",a:["Any number of categories","Exactly two binary values","Continuous values","Missing values"],c:1,e:"Binarization converts values into 0 and 1."},{q:"Which method is suitable when you already know meaningful age boundaries?",a:["K-Means","Equal Width","Custom Binning","Power Transformation"],c:2,e:"Custom Binning uses domain knowledge to define intervals."},{q:"Does binning automatically remove outliers?",a:["Yes","No"],c:1,e:"Binning changes the representation of observations; it does not remove them."},{q:"What does encode='ordinal' produce?",a:["Separate binary columns","Integer bin labels","Text categories","Missing values"],c:1,e:"Ordinal encoding represents bins using integer labels."},{q:"What should be used on test data after fitting KBinsDiscretizer on training data?",a:["fit_transform()","fit()","transform()","fit_predict()"],c:2,e:"Reuse the training bin boundaries with transform()."}]}
{@body const [answers,setAnswers] = DIL.useState({})}
{@body const [submitted,setSubmitted] = DIL.useState(false)}

Lecture 32 quiz
[{Object.keys(answers).length}/{questions.length} answered]

{#each questions as q,i}

{i+1}. {q.q}
setAnswers(p=>({...p,[i]:Number(v)}))}>
{#each q.a as option,j}
{option}
{/each}

{#if submitted}

{#if answers[i]===q.c}

Correct
{:else}

Correct answer: {q.a[q.c]}
{/if}

{q.e}

{/if}

{#if i!==questions.length-1}

{/if}
{/each}
{#if submitted}
{questions.filter((q,i)=>answers[i]===q.c).length}/{questions.length} correct
{setAnswers({});setSubmitted(false)}}>Retry quiz
{:else}
setSubmitted(true)}>Submit answers
{/if}

Final Understanding

The entire lecture revolves around one central concept:

We can change the representation of numerical features by dividing them into groups or converting them into binary values.

Remember these differences:

- Equal Width: Same width.
- Equal Frequency: Same number of observations.
- K-Means: Groups based on similarity.
- Custom Binning: Boundaries selected using domain knowledge.
- Binarization: Only 0 and 1.

The most important Scikit-learn classes are `KBinsDiscretizer` and `Binarizer`.

Lecture 32 is covered from the uploaded transcript, including its practical experiments.

Next: **Lecture 33**, continuing in the same detailed, textbook-style format.

CHATGPT (Message 14)

PART 12 — IMPORTANT COMPARISONS

13. Discretization vs Binarization

This is a common conceptual question.

Property	Discretization	Binarization
Meaning	Converts numerical values into intervals	Converts numerical values into two categories
Output categories	Multiple	Exactly two
Output example	0, 1, 2, 3	0, 1

| Example | Age groups | Travelling alone or not |
| Scikit-learn | `KBinsDiscretizer` | `Binarizer` |

Remember:

Binning can create multiple categories. Binarization creates only two categories.

14. Complete Binning Strategy Comparison

Technique	Main principle	Best situation
Equal Width	Equal interval widths	Data with manageable numerical ranges
Equal Frequency	Equal observations per bin	Skewed distributions
K-Means	Similarity-based clusters	Naturally separated groups
Custom Binning	Manually selected boundaries	Domain knowledge or business rules
Decision Tree Binning	Target-based splitting	Supervised feature discretization

One important clarification: Decision Tree Binning is mentioned in the lecture but its implementation is not covered in detail.

PART 13 — COMPLETE CODE REVISION

Here is a consolidated code reference for the entire lecture.

You can save this section separately for quick revision.

15. Equal Width Binning

```
from sklearn.preprocessing import KBinsDiscretizer

kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='uniform'
)

X_binned = kbd.fit_transform(X_train)
```

16. Equal Frequency Binning

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='quantile'
)

X_binned = kbd.fit_transform(X_train)
```

17. K-Means Binning

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='kmeans'
)

X_binned = kbd.fit_transform(X_train)
```

18. One-Hot Binning

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='onehot',
strategy='quantile'
)

X_binned = kbd.fit_transform(X_train)
```

This produces a sparse matrix by default.

If you specifically need a dense array:

```
kbd = KBinsDiscretizer(
n_bins=5,
encode='onehot-dense',
strategy='quantile'
)
```

19. Custom Binning

```
import pandas as pd

df['Age_Category'] = pd.cut(
df['Age'],
bins=[0, 18, 60, 100],
labels=['Minor', 'Adult', 'Senior'],
include_lowest=True
)
```

20. Binarization

```
from sklearn.preprocessing import Binarizer

bn = Binarizer(
threshold=5,
copy=True
)

X_binary = bn.fit_transform(X)
```

21. Pipeline Implementation

A useful way to combine binning and a model is:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.tree import DecisionTreeClassifier

pipe = Pipeline([
('binning', KBinsDiscretizer(
n_bins=5,
encode='ordinal',
strategy='quantile'
)),
('model', DecisionTreeClassifier())
])

pipe.fit(X_train, y_train)

y_pred = pipe.predict(X_test)
```

This ensures the bin boundaries are learned from the training data and reused during prediction.

PART 14 — ADDITIONAL CONCEPTS TO UNDERSTAND

These points help connect the lecture to actual Machine Learning projects.

22. Does binning always improve model performance?

No.

Suppose you have a feature:

$$
Age=[21,22,23,24,25]
$$

If you convert all these values into one category called Young, you lose the difference between 21 and 25.

The original numerical information becomes less detailed.

This is called information loss.

Binning can sometimes improve performance, but it can also decrease performance.

It depends on:

- The dataset.
- The algorithm.
- The number of bins.
- The binning strategy.
- The relationship between the feature and the target.

23. Does binning remove outliers?

No.

This is an important distinction.

Suppose:

$$
X=[10,12,15,20,25,500]
$$

After binning, 500 might belong to the final bin.

But the original observation still exists.

We have only changed its representation.

Binning does not delete the outlier.

24. Does Ordinal Encoding introduce an ordering problem?

It can.

Suppose you convert Age into:

- Young = 0
- Adult = 1
- Senior = 2

These categories have a meaningful order.

But imagine you have:

- Red = 0
- Blue = 1
- Green = 2

There is no natural numerical order between these colors.

Assigning integer values can create an artificial relationship.

Therefore, One-Hot Encoding may be more appropriate for categories without a meaningful order.

For bins, ordinal encoding can be reasonable because the intervals have a natural order. However, the downstream algorithm matters.

25. Important implementation precautions

- Fit your binning transformer on training data only.
- Apply the learned boundaries to testing data using `transform()`.
- Check whether duplicate values create fewer distinct bins than requested.

- Choose the number of bins carefully.
- Compare performance before and after binning.
- Remember that ordinal bin labels are category identifiers, not necessarily the original numerical values.

PART 15 — QUICK REVISION NOTES

Lecture 32 — Cheat Sheet
1. Discretization

Converts continuous numerical values into discrete intervals.

2. Equal Width Binning

$$
Width=\frac{Max-Min}{K}
$$

- Equal interval width.
- Unequal frequency is possible.
- Scikit-learn strategy: `uniform`.

3. Equal Frequency Binning

- Approximately equal observations per bin.
- Unequal interval width.
- Based on quantiles.
- Scikit-learn strategy: `quantile`.

4. K-Means Binning

- Uses K-Means clustering.
- Assigns observations to clusters.
- Each cluster becomes a bin.
- Scikit-learn strategy: `kmeans`.

5. Custom Binning

- Manually selected intervals.
- Uses domain knowledge.
- Pandas: `pd.cut()`.

6. Binarization

Converts numerical data into 0 and 1.

- Uses a threshold.
- Scikit-learn: `Binarizer`.

7. Main Scikit-learn classes

KBinsDiscretizer()
Binarizer()

8. KBinsDiscretizer parameters

n_bins
strategy
encode

9. Binarizer parameters

threshold
copy

PART 16 — TEST YOUR UNDERSTANDING

Try these questions after completing the chapter.

{@body const questions = [{q:"What is Discretization?",a:["Converting numerical values into discrete intervals","Removing all outliers","Scaling data between 0 and 1","Converting text into numbers"],c:0,e:"Discretization divides continuous numerical values into a limited number of groups."},{q:"In Equal Width Binning, what remains equal?",a:["Number of observations","Interval width","Mean of each bin","Number of outliers"],c:1,e:"Each interval has the same numerical width, but frequencies may differ."},{q:"In Equal Frequency Binning, what remains approximately equal?",a:["Bin width","Number of observations per bin","Maximum value","Minimum value"],c:1,e:"Quantile Binning aims to put approximately equal numbers of observations in each bin."},{q:"Which strategy is used for K-Means Binning in Scikit-learn?",a:["uniform","quantile","kmeans","custom"],c:2,e:"Use strategy='kmeans' in KBinsDiscretizer."},{q:"Which class is used for discretization?",a:["Binarizer","KBinsDiscretizer","StandardScaler","SimpleImputer"],c:1,e:"KBinsDiscretizer performs numerical feature discretization."},{q:"What is the output of Binarization?",a:["Any number of categories","Exactly two binary values","Continuous values","Missing values"],c:1,e:"Binarization converts values into 0 and 1."},{q:"Which method is suitable when you already know meaningful age boundaries?",a:["K-Means","Equal Width","Custom Binning","Power Transformation"],c:2,e:"Custom Binning uses domain knowledge to define intervals."},{q:"Does binning automatically remove outliers?",a:["Yes","No"],c:1,e:"Binning changes the representation of observations; it does not remove them."},{q:"What does encode='ordinal' produce?",a:["Separate binary columns","Integer bin labels","Text categories","Missing values"],c:1,e:"Ordinal encoding represents bins using integer labels."},{q:"What should be used on test data after fitting KBinsDiscretizer on training data?",a:["fit_transform()","fit()","transform()","fit_predict()"],c:2,e:"Reuse the training bin boundaries with transform()."}]}
{@body const [answers,setAnswers] = DIL.useState({})}
{@body const [submitted,setSubmitted] = DIL.useState(false)}

Lecture 32 quiz
[{Object.keys(answers).length}/{questions.length} answered]

{#each questions as q,i}

{i+1}. {q.q}

setAnswers(p=>({...p,[i]:Number(v)}))}>
{#each q.a as option,j}
{option}
{/each}

{#if submitted}

{#if answers[i]===q.c}

Correct
{:else}

Correct answer: {q.a[q.c]}
{/if}

{q.e}

{/if}

{#if i!==questions.length-1}

{/if}
{/each}
{#if submitted}
{questions.filter((q,i)=>answers[i]===q.c).length}/{questions.length} correct
{setAnswers({});setSubmitted(false)}}>Retry quiz
{:else}
setSubmitted(true)}>Submit answers
{/if}

Final Understanding

The entire lecture revolves around one central concept:

We can change the representation of numerical features by dividing them into groups or converting them into binary values.

Remember these differences:

- Equal Width: Same width.
- Equal Frequency: Same number of observations.
- K-Means: Groups based on similarity.
- Custom Binning: Boundaries selected using domain knowledge.
- Binarization: Only 0 and 1.

The most important Scikit-learn classes are `KBinsDiscretizer` and `Binarizer`.

Lecture 32 is covered from the uploaded transcript, including its practical experiments.

Next: **Lecture 33**, continuing in the same detailed, textbook-style format.

[Attachment: Handling Mixed Variables Feature Engineering.txt (26545 bytes)]

CHATGPT (Message 15)

Lecture 33 — Handling Mixed Variables | Feature Engineering

[Complete transcript covered] Based on the uploaded 12-minute lecture transcript

This lecture explains how to handle **mixed variables** in a dataset. The central problem is that a single column may contain both numerical and categorical information. Before applying machine learning algorithms, we need to separate these two types of information into appropriate columns.

The instructor demonstrates two common situations:

1. A single cell contains numerical and categorical information together.
2. A single column contains numerical values in some rows and categorical values in other rows.

Let's understand both cases step by step.

---

1. What is mixed data?

In machine learning, a dataset may contain different types of information:

- **Numerical data:** Values representing quantities, such as age, salary, number of passengers, or ticket numbers.
- **Categorical data:** Values representing categories, such as gender, ticket class, city, or passenger type.

Usually, these are stored in separate columns.

However, real-world datasets are not always clean. Sometimes a column contains both types of information.

For example:

Passenger	Cabin
A	C85
B	B57
C	D123
D	45
E	A23

Here, the `Cabin` column contains information in a mixed format.

The value `C85` contains two pieces of information:

- `C` represents the cabin category or deck.
- `85` represents the cabin number.

These two pieces of information have different meanings and should be stored separately.

The objective of handling mixed variables is to extract the different types of information and store them in appropriate columns.

---

2. Two common types of mixed data

The instructor explains two situations in which mixed data can appear.

Situation 1: Numerical and categorical information in the same cell

Consider the following example:

Cabin
C85
B57
D123
A45

Each value contains both a categorical and a numerical component.

For example:

- `C85`: Category = C, Number = 85
- `B57`: Category = B, Number = 57
- `D123`: Category = D, Number = 123
- `A45`: Category = A, Number = 45

We should separate the information into two columns.

The resulting dataset becomes:

Cabin_Category	Cabin_Number
C	85
B	57
D	123
A	45

Now both columns have a consistent data type.

- `Cabin_Category` contains categorical information.
- `Cabin_Number` contains numerical information.

This makes the dataset easier to process and allows us to apply suitable preprocessing techniques to each column.

General approach

Whenever a cell contains both numerical and categorical information:

1. Identify the categorical component.
2. Identify the numerical component.
3. Create two new columns.
4. Extract the corresponding information.
5. Handle missing values separately in both columns.

---

3. Situation 2: Numerical and categorical values in different rows

This situation is slightly different.

Consider a column containing:

Number
5
3
ML
2
AI
7
Data

Here, the column contains numerical values in some rows and categorical values in other rows.

The numerical and categorical values are not combined inside individual cells. Instead, they are mixed across different rows.

We cannot treat the entire column as a purely numerical feature or a purely categorical feature.

The solution is again to create two separate columns.

Original	Numerical	Categorical
5	5	NaN
3	3	NaN
ML	NaN	ML
2	2	NaN
AI	NaN	AI
7	7	NaN
Data	NaN	Data

Notice that:

- Numerical values go into the numerical column.
- Categorical values go into the categorical column.
- The other column receives a missing value (`NaN`).

This preserves the original information without mixing the two data types.

---

4. Practical implementation using Pandas

The instructor creates a small dataset based partly on the Titanic dataset and adds a few artificial values to demonstrate mixed-variable handling.

The dataset contains four columns:

- `Cabin`
- `Ticket`
- `Number`
- `Survived`

The `Survived` column is the output or target variable.

The `Cabin` column contains values such as `C85`, `B57`, and `D123`.

The `Number` column represents the number of people travelling with a passenger. Some values are numerical, while others represent categories such as `Alone`.

Let's implement the techniques.

4.1 Separate numerical and categorical values

First, create two new columns:

```
df['num'] = pd.to_numeric(df['Number'], errors='coerce')

df['cat'] = np.where(
df['num'].isnull(),
df['Number'],
np.nan
)
```

Let's understand each part.

Step 1: `pd.to_numeric()`

```
df['num'] = pd.to_numeric(df['Number'], errors='coerce')
```

This attempts to convert the values in the `Number` column into numerical values.

The parameter:

errors='coerce'

means:

- If a value can be converted into a number, convert it.
- If it cannot be converted, replace it with `NaN`.

For example:

Original	After conversion
5	5
3	3
Alone	NaN
2	2
7	7

Step 2: `np.where()`

```
df['cat'] = np.where(
df['num'].isnull(),
df['Number'],
np.nan
)
```

This creates the categorical column.

The condition:

```
df['num'].isnull()
```

checks whether the numerical conversion produced a missing value.

- If the numerical value is missing, take the original value from `Number`.
- Otherwise, store `NaN`.

The result is:

Original	num	cat
5	5	NaN
3	3	NaN
Alone	NaN	Alone
2	2	NaN
7	7	NaN

Important: The original column is retained in this example. Once you have verified the extracted information, you can decide whether to remove the original mixed column.

4.2 Why is this useful?

Suppose you want to apply preprocessing to your dataset.

You might want to:

- Apply numerical imputation to `num`.
- Apply categorical imputation to `cat`.
- Scale numerical features.
- Encode categorical features.

You cannot correctly apply the same preprocessing operation to both types of information.

Separating them makes the preprocessing pipeline much easier to construct.

---

5. Handling mixed Cabin values

Now let's understand the first situation in more detail.

Consider these values:

Cabin
C85
B57
D123
A45
C23

We want to extract:

- The cabin category.
- The cabin number.

The instructor demonstrates using regular expressions.

5.1 Extract numerical information

The objective is to capture the numerical part of the cabin value.

Conceptually:

```
df['cabin_num'] = df['Cabin'].str.extract(
r'(\d+)'
)
```

Explanation:

- `str.extract()` extracts a matching pattern from a string.
- `\d` represents a digit.
- `+` means one or more occurrences.
- `(\d+)` captures the sequence of digits.

Example:

Cabin	cabin_num
C85	85
B57	57
D123	123
A45	45
C23	23

5.2 Extract categorical information

Now we extract the categorical part.

df['cabin_cat] = df['Cabin'].str[0]

This takes the first character of each cabin value.

Result:

Cabin	cabin_cat
C85	C
B57	B
D123	D
A45	A
C23	C

The original mixed column has now been separated into two features.

Original feature

`Cabin = C85`

Categorical

C
Cabin category

Numerical

85
Cabin number

Note: The transcript's regular-expression code is not perfectly captured by automatic transcription. The code above expresses the extraction logic demonstrated in the lecture, using a clear Pandas implementation.

---

6. Handling missing values after separation

After separating the mixed column, we may have missing values in both new columns.

For example:

Original	Numerical	Categorical
5	5	NaN
Alone	NaN	Alone
3	3	NaN
7	7	NaN

The instructor discusses filling these missing values.

Numerical column

In the Titanic example, the numerical column represents the number of people travelling with a passenger.

If the categorical value is `Alone`, the numerical value can logically be represented as `0`.

Therefore:

df['num'] = df['num'].fillna(0)

Categorical column

For the categorical column, we can use a placeholder such as `Not` or `Missing`.

For example:

df['cat'] = df['cat'].fillna('Not')

Now the resulting dataset looks like:

Original	Numerical	Categorical
5	5	Not
Alone	0	Alone

| 3 | 3 | Not |
| 7 | 7 | Not |

The important idea is that missing values should be handled according to the meaning and type of each feature.

A caution: Filling numerical missing values with zero is appropriate only when zero has a meaningful interpretation in that feature. It should not be used automatically for every numerical column.

---

7. Handling mixed Ticket values

The instructor also demonstrates the same general idea with the Titanic `Ticket` column.

Ticket values can contain both letters and numbers.

For example, a ticket might contain:

- A categorical prefix.
- A numerical ticket identifier.

The instructor extracts these into two separate columns:

1. `ticket_num`
2. `ticket_cat`

The exact ticket formats can be more complicated than cabin values. Some tickets contain multiple letters, spaces, punctuation, or no clear numerical component.

The lecture demonstrates using regular expressions to extract the relevant parts. It does not explain every possible ticket-format pattern in detail.

The main objective is to separate the categorical and numerical information rather than treat the complete ticket string as one feature.

---

8. Why do we need to handle mixed variables?

Consider what happens if we leave a mixed column unchanged.

A numerical preprocessing operation may fail because some values are strings.

Similarly, categorical encoding may create unnecessary categories if numerical values are treated as strings.

After separating the information, we can apply appropriate operations independently.

Feature type	Possible preprocessing
Numerical	Missing-value imputation

Numerical	Scaling
Numerical	Outlier treatment
Categorical	Missing-value imputation
Categorical	One-hot encoding
Categorical	Ordinal encoding, where appropriate

For example, after extracting `cabin_num` and `cabin_cat`:

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

You can build separate numerical and categorical preprocessing pipelines.

This is particularly useful when working with `ColumnTransformer` and `Pipeline`.

---

9. Complete conceptual summary of the lecture

The instructor's main message is that mixed data can appear in two common forms.

Type 1 — Both values inside one cell

Example: Cabin = C85

C85

C

Category

85

Number

Type 2 — Different data types in different rows

Example: 5, 3, Alone, 2, 7

`5 → 3 → Alone → 2 → 7`

Numerical

5, 3, NaN, 2, 7

Categorical

NaN, NaN, Alone, NaN, NaN

The process is:

1. Identify the mixed feature.
2. Understand what information it contains.
3. Separate numerical and categorical components.
4. Create separate columns.
5. Handle missing values independently.
6. Apply suitable preprocessing to each column.

---

10. Important code to revise

```
import pandas as pd
import numpy as np
```

Situation 1: Numerical and categorical values in different rows

```
df['num'] = pd.to_numeric(
df['Number'],
errors='coerce'
)

df['cat'] = np.where(
df['num'].isnull(),
df['Number'],
np.nan
)
```

Handle missing values

```
df['num'] = df['num'].fillna(0)
df['cat'] = df['cat'].fillna('Not')
```

Situation 2: Numerical and categorical information in one cell

Extract numerical part

df['cabin_num'] = df['Cabin'].str.extract(r'(\d+)')

Extract categorical part
df['cabin_cat'] = df['Cabin'].str[0]

---

11. What this lecture does not cover

The transcript focuses on the two mixed-data situations and their separation. It does not provide a complete discussion of:

- All possible regular-expression patterns for complex ticket formats.
- How to select the best imputation strategy statistically.
- How to build a complete `ColumnTransformer` for the extracted features.
- How to evaluate whether the extracted features improve model performance.

Those are useful next steps, but they should not be confused with the material actually demonstrated in this lecture.

---

12. Quick revision questions

Try answering these without looking at the notes.

{@body const questions = [{q:"What is mixed data in the context of this lecture?",o:["A dataset containing only missing values","A feature containing numerical and categorical information","A dataset containing only strings"],a:1,e:"Mixed data means numerical and categorical information appears together, either in the same cell or across different rows."},{q:"A cabin value is C85. What information can be extracted?",o:["Only 85","Only C","C as category and 85 as number"],a:2,e:"The letter represents the categorical component and the digits represent the numerical component."},{q:"What does errors='coerce' do in pd.to_numeric()?",o:["Converts invalid values to NaN","Deletes invalid rows","Converts every value to zero"],a:0,e:"Values that cannot be converted to numbers become NaN."},{q:"Why create separate numerical and categorical columns?",o:["To increase the number of rows","To apply suitable preprocessing to each data type","To remove all missing values automatically"],a:1,e:"Numerical and categorical features often require different preprocessing techniques."},{q:"In the lecture's Alone example, why can the numerical value be zero?",o:["Because zero is always the best imputation value","Because the passenger is travelling with zero companions","Because the original value was missing"],a:1,e:"Zero has a specific meaning here: the passenger is travelling alone."}]}
{@body const [answers,setAnswers] = DIL.useState({})}
{@body const [submitted,setSubmitted] = DIL.useState(false)}

Test your understanding
[{Object.keys(answers).length}/{questions.length} answered]

{#each questions as q,i}

{i+1}. {q.q}
setAnswers(p=>({...p,[i]:Number(v)}))}>
{#each q.o as option,j}

{option}
{/each}

{#if submitted}

{#if answers[i]===q.a}

Correct
{:else}

Correct answer: {q.o[q.a]}
{/if}

{q.e}

{/if}

{#if i!==questions.length-1}

{/if}
{/each}
{#if submitted}
{questions.filter((q,i)=>answers[i]===q.a).length} / {questions.length}
{setAnswers({});setSubmitted(false)}}>Try again
{:else}
setSubmitted(true)}>Check answers
{/if}

Final takeaway: Never blindly treat a mixed column as purely numerical or purely categorical. First understand what information is present, separate it into meaningful features, and then preprocess each feature according to its data type.

Lecture 33 is complete. The next lecture in your sequence is about handling date and time columns, as mentioned at the end of this transcript.

[Attachment: Handling Date and Time Variables Day 34 100 Days of Machine Learning.txt (31897 bytes)]

CHATGPT (Message 16)

Lecture 34 — Handling Date and Time Variables

100 Days of Machine Learning · Feature Engineering

[Complete transcript covered]

This lecture explains how to extract useful information from date and time columns using Pandas.

The main idea is that **a single date or time value contains multiple pieces of information**. Instead of treating it as one ordinary column, we can extract its individual components and use them as separate machine learning features.

For example, consider:

```
2019-12-12 15:30:45
```

This single value contains:

- Year: 2019
- Month: 12
- Day: 12
- Hour: 15
- Minute: 30
- Second: 45
- Day of the week
- Week number
- Quarter of the year

These extracted features can help a machine learning model identify patterns that might otherwise remain hidden.

---

1. Why are date and time features important?

Date and time information can be extremely useful in machine learning projects.

Consider an expense-tracking application.

Suppose you have the following data:

Date	Expense
2024-01-01	500
2024-01-02	200
2024-01-06	900
2024-02-01	400
2024-02-05	700

You may want to answer questions such as:

- On which day of the week does the user spend the most?
- Does the user spend more money on weekends?
- Which month has the highest expenses?
- Are expenses higher during a particular quarter?
- Is there a difference between weekday and weekend spending?

A raw date column does not directly expose all these patterns.

We can extract additional features from it and use them in our analysis.

The same concept applies to:

- E-commerce purchases.
- Website traffic.
- Banking transactions.
- Chat applications.
- Sales forecasting.
- Customer activity analysis.

---

2. The first step: Convert the column into datetime format

This is the most important step in the lecture.

When you load a CSV file using Pandas, date and time columns are often imported as strings or `object` data types.

For example:

import pandas as pd

df = pd.read_csv('orders.csv')

df.info()

You might see:

order_date object

The problem is that Pandas does not automatically understand this column as a proper datetime column.

It treats the values as ordinary objects.

Consequently, you cannot directly use all the datetime-specific properties and operations.

Convert using `pd.to_datetime()`

df['date'] = pd.to_datetime(df['date'])

Now check:

df.info()

The data type should become something like:

date datetime64[ns]

Now Pandas recognizes the column as datetime data.

Remember this conversion

`object` / string

"2019-12-12"

`datetime64[ns]`

Pandas understands it as a date

Important: If your date strings have inconsistent or invalid formats, conversion may require additional handling. You can use:

pd.to_datetime(df['date'], errors='coerce')

Invalid date values will become `NaT` (Not a Time), which is the datetime equivalent of a missing value.

---

3. Understanding `.dt` — the datetime accessor

Once the column is converted into datetime format, Pandas provides a special accessor called `.dt`.

It allows us to extract different components from datetime values.

The basic syntax is:

df['date'].dt.property

For example:

df['date'].dt.year

This extracts the year from each date.

The lecture uses two datasets:

1. `orders.csv` — for date-related operations.
2. `messages.csv` — for time-related operations.

Let's first understand the date operations.

---

4. Extracting information from dates

Suppose we have:

df['date']

Date
2019-12-12
2018-08-08
2019-10-10
2018-01-15

4.1 Extract the year

df['year'] = df['date'].dt.year

Output:

Date	Year
2019-12-12	2019
2018-08-08	2018
2019-10-10	2019
2018-01-15	2018

This feature can help identify yearly trends.

For example, you might want to compare sales between 2018 and 2019.

4.2 Extract the month number

df['month'] = df['date'].dt.month

Output:

Date	Month
2019-12-12	12
2018-08-08	8
2019-10-10	10
2018-01-15	1

The month is represented as an integer from 1 to 12.

4.3 Extract the month name

Sometimes the month number is not sufficient. You may want the actual month name.

df['month_name'] = df['date'].dt.month_name()

Output:

Date	Month Name
2019-12-12	December
2018-08-08	August
2019-10-10	October

| 2018-01-15 | January |

This is useful for visualization and categorical analysis.

4.4 Extract the day of the month

df['day'] = df['date'].dt.day

Output:

Date	Day
2019-12-12	12
2018-08-08	8
2019-10-10	10
2018-01-15	15

This tells us which day of the month the event occurred.

Be careful: `dt.day` does not give the day of the year or day of the week. It gives the day within the month.

---

5. Extracting the day of the week

This is an important feature when working with time-based datasets.

For example, in an e-commerce dataset, sales might be higher on Saturdays and Sundays than on weekdays.

Pandas provides two useful properties.

5.1 Day of the week as a number

df['day_of_week'] = df['date'].dt.dayofweek

The values follow this convention:

Day	Value
Monday	0
Tuesday	1
Wednesday	2
Thursday	3
Friday	4
Saturday	5
Sunday	6

Important: Monday starts at zero and Sunday is six.

The lecture notes that the numbering may look different from what you initially expect. The important thing is that the numbering is consistent.

5.2 Day of the week as a name

df['day_name'] = df['date'].dt.day_name()

Output:

Date	Day Name
2019-12-12	Thursday
2018-08-08	Wednesday
2019-10-10	Thursday
2018-01-15	Monday

Use `dayofweek` when you need a numerical representation and `day_name()` when you want the textual name.

---

6. Checking whether a date falls on a weekend

Sometimes you do not need the exact weekday. You only want to know whether a date is a weekend.

For example:

- 0 = Not a weekend
- 1 = Weekend

First, extract the weekday number:

```
df['day_of_week'] = df['date'].dt.dayofweek
```

Then create a weekend feature:

```
df['is_weekend'] = (
df['day_of_week'] >= 5
).astype(int)
```

The logic is:

- Monday to Friday: values 0 to 4.
- Saturday and Sunday: values 5 and 6.

Therefore:

```
df['day_of_week'] >= 5
```

returns `True` for weekends and `False` for weekdays.

Converting to integers gives:

- `True` → 1
- `False` → 0

Example:

Day	Day Number	Is Weekend
Monday	0	0
Tuesday	1	0
Friday	4	0
Saturday	5	1
Sunday	6	1

This is a useful binary feature for machine learning.

---

7. Extracting the week number

We can also determine which week of the year a particular date belongs to.

The lecture demonstrates extracting the week number using:

df['week'] = df['date'].dt.isocalendar().week

For example:

Date	Week
2019-12-12	50
2018-08-08	32

The week number tells us where the date falls within the calendar year according to the ISO week-date system.

A small technical clarification: ISO week numbering can differ from simple calendar-year week counting near the beginning and end of a year. The ISO system starts weeks on Monday and assigns week 1 according to its own rules.

---

8. Extracting the quarter of the year

A year has four quarters.

Each quarter contains three months.

Quarter	Months
Q1	January, February, March
Q2	April, May, June
Q3	July, August, September
Q4	October, November, December

We can extract the quarter directly:

df['quarter'] = df['date'].dt.quarter

Example:

Date	Month	Quarter
2019-12-12	12	4
2018-08-08	8	3
2019-10-10	10	4
2018-01-15	1	1

This is useful when analyzing quarterly sales, revenue, expenses, or business performance.

---

9. Creating a semester feature

The instructor also demonstrates deriving a semester from the quarter.

For the purpose of this lecture, the year is divided into two semesters:

- First semester: Q1 and Q2.
- Second semester: Q3 and Q4.

The logic is:

```
df['semester'] = np.where(
df['quarter'].isin([1, 2]),
1,
2
)
```

This creates:

Quarter	Semester
1	1
2	1
3	2
4	2

Notice that this is a custom semester definition used in the lecture.

In real applications, semester definitions can depend on the academic calendar or financial year. Always use the definition relevant to your problem.

---

10. Calculating the time difference between two dates

Another important operation is calculating the difference between two dates.

Suppose we have:

- A transaction date.
- Today's date.

We want to know how much time has passed since the transaction.

First, import the datetime module:

```
import datetime
```

Get today's date:

```
today = datetime.datetime.today()
```

Now calculate the difference:

```
df['date_difference'] = today - df['date']
```

The result is a Pandas timedelta value.

For example:

```
16 days 13:20:30
```

This means the difference is 16 days, 13 hours, 20 minutes, and 30 seconds.

10.1 Extract only the number of days

```
df['days_difference'] = (
today - df['date']
).dt.days
```

Now you get a numerical value representing the number of whole days.

Example:

Transaction Date	Days Difference
2024-01-01	20
2024-01-05	16
2024-01-10	11

The actual values depend on the current date when the code runs.

10.2 Convert the difference into months

The instructor also demonstrates converting the difference into months.

The basic idea is:

1. Calculate the difference between today's date and the given date.
2. Convert the difference into a suitable numerical unit.
3. Divide by the approximate number of days in a month.

For example:

```
df['month_difference'] = (
(today - df['date']) / np.timedelta64(1, 'M')
)
```

This gives an approximate number of months.

Important: This is not an exact calendar-month calculation. Months have different lengths. If your application needs exact calendar-month differences, use calendar-aware date arithmetic rather than assuming every month has the same duration.

Part 2 — Handling Time Variables

The second half of the lecture focuses on time-related columns.

The instructor uses a messaging dataset containing timestamps of messages.

A time value may look like:

`15:30:45`

It contains:

- Hour = 15
- Minute = 30
- Second = 45

We can extract each component independently.

11. Convert a time column into datetime format

Just as we converted date columns, we need to convert the time-related column.

For example:

```
df['date'] = pd.to_datetime(df['date'])
```

The lecture demonstrates converting the timestamp column using `pd.to_datetime()` and then accessing its datetime properties.

If the column contains complete timestamps, the resulting column can be used to extract both date and time information.

For a column containing only time strings, parsing may require a suitable time format or combining the time with a date.

---

12. Extracting hour, minute and second

Suppose we have:

Timestamp
15:30:45
09:20:10
18:45:30
12:15:05

After conversion to datetime format, use `.dt`.

12.1 Extract hour

df['hour'] = df['date'].dt.hour

Output:

Timestamp	Hour
15:30:45	15
09:20:10	9
18:45:30	18
12:15:05	12

12.2 Extract minute

df['minute'] = df['date'].dt.minute

Output:

Timestamp	Minute
15:30:45	30
09:20:10	20
18:45:30	45
12:15:05	15

12.3 Extract second

df['second'] = df['date'].dt.second

Output:

Timestamp	Second
15:30:45	45
09:20:10	10
18:45:30	30
12:15:05	5

We have now extracted three independent features from a single timestamp.

---

13. Calculating the difference between two time values

The lecture also discusses calculating time differences.

Suppose a user sends two messages:

- First message: 10:00:00
- Second message: 10:30:00

We want to calculate how much time passed between the messages.

The general operation is subtraction:

time_difference = time2 - time1

For Pandas datetime columns, you can use:

```
df['time_difference'] = (
df['time2'] - df['time1']
)
```

The result is a timedelta.

For example:

0 days 00:30:00

This means 30 minutes have passed.

13.1 Convert the difference into seconds

```
df['seconds_difference'] = (
df['time_difference'].dt.total_seconds()
)
```

Example:

Time Difference	Seconds
00:01:00	60
00:05:00	300
00:30:00	1800
01:00:00	3600

13.2 Convert the difference into minutes

```
df['minutes_difference'] = (
df['time_difference'].dt.total_seconds() / 60
)
```

13.3 Convert the difference into hours

```
df['hours_difference'] = (
df['time_difference'].dt.total_seconds() / 3600
)
```

This is useful when measuring:

- Time between messages.
- Time between customer visits.
- Time between orders.
- Duration of user sessions.
- Time taken to complete a task.

Remember: `.dt.total_seconds()` is useful because it converts the complete timedelta into one numerical quantity. It includes days, hours, minutes and seconds rather than returning only one component.

---

14. Combining date and time information

Sometimes a dataset contains separate date and time columns.

For example:

Date	Time
2024-01-01	10:30:00
2024-01-02	12:15:00

You may want to combine them into a single timestamp.

One approach is:

```
df['datetime'] = pd.to_datetime(
df['Date'] + ' ' + df['Time']
)
```

Now you have a complete datetime column.

You can extract:

```
df['year'] = df['datetime'].dt.year
df['month'] = df['datetime'].dt.month
df['day'] = df['datetime'].dt.day

df['hour'] = df['datetime'].dt.hour
df['minute'] = df['datetime'].dt.minute
df['second'] = df['datetime'].dt.second
```

This is an additional practical application of the lecture's datetime conversion and extraction techniques.

---

15. Complete Pandas datetime reference

This is the most important section to keep for offline revision.

Date extraction

Operation	Pandas code
Year	`df['date'].dt.year`
Month number	`df['date'].dt.month`
Month name	`df['date'].dt.month_name()`
Day of month	`df['date'].dt.day`
Day of week number	`df['date'].dt.dayofweek`
Day name	`df['date'].dt.day_name()`
Week number	`df['date'].dt.isocalendar().week`
Quarter	`df['date'].dt.quarter`
Weekend check	`df['date'].dt.dayofweek >= 5`

Time extraction

Operation	Pandas code
Hour	`df['date'].dt.hour`
Minute	`df['date'].dt.minute`
Second	`df['date'].dt.second`

Date and time calculations

Operation	Code
Difference between dates	`date2 - date1`
Difference in days	`difference.dt.days`
Difference in total seconds	`difference.dt.total_seconds()`
Difference in minutes	`difference.dt.total_seconds() / 60`
Difference in hours	`difference.dt.total_seconds() / 3600`

---

16. Complete practical code

The following example combines the most important operations from the lecture into one workflow.

```
import pandas as pd
import numpy as np
import datetime
```

Load dataset

```
df = pd.read_csv('orders.csv')
```

Convert date column to datetime

```
df['date'] = pd.to_datetime(
df['date'],
errors='coerce'
)
```

Extract date components

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['month_name'] = df['date'].dt.month_name()

df['day'] = df['date'].dt.day
df['day_of_week'] = df['date'].dt.dayofweek
df['day_name'] = df['date'].dt.day_name()
```

Weekend feature

```
df['is_weekend'] = (
df['day_of_week'] >= 5
).astype(int)
```

Week and quarter

```
df['week'] = df['date'].dt.isocalendar().week
df['quarter'] = df['date'].dt.quarter
```

Semester feature

```
df['semester'] = np.where(
df['quarter'].isin([1, 2]),
1,
2
)
```

Difference from today's date

```
today = pd.Timestamp.now()

df['date_difference'] = today - df['date']
```

Difference in days

```
df['days_difference'] = (
df['date_difference'].dt.days
)
```

Display result

```
print(df.head())
```

This example gives you a complete date-feature extraction workflow.

---

17. Important additional concepts

These are useful clarifications for applying the lecture in actual ML projects.

A. Date components are not always ordinary numerical features

For example, month values are:

`1, 2, 3, ..., 12`

But December and January are adjacent in the calendar even though their numerical values are far apart.

Similarly, hours are cyclical:

- 23:00 is close to 00:00.
- But numerically, 23 and 0 are far apart.

For some machine learning models, cyclical encoding using sine and cosine can represent this relationship more appropriately.

For example:

```
df['month_sin'] = np.sin(
2 *np.pi* df['month'] / 12
)

df['month_cos'] = np.cos(
2 *np.pi* df['month'] / 12
)
```

This is additional ML context, not a technique demonstrated in the lecture.

B. Be careful with time zones

A timestamp such as `10:00` does not necessarily represent the same moment everywhere.

If your data comes from different countries or servers, time zones matter.

Pandas supports timezone-aware datetime values. The lecture briefly mentions that timezone handling may occasionally be necessary, but does not teach the detailed implementation.

C. Avoid data leakage

Suppose you are predicting whether a customer will purchase a product tomorrow.

You must not use information that becomes available only after tomorrow's prediction point.

Similarly, when constructing time-based training and testing datasets, maintain the chronological order when appropriate.

---

18. Quick revision — What you must remember

The complete workflow

Load your dataset using Pandas.
Check the column's data type using `df.info()`.

Convert the date column using `pd.to_datetime()`.
Use `.dt` to extract useful components.
Create new features such as year, month, day, weekday, quarter and hour.
Calculate time differences using subtraction.
Convert timedelta values into days, seconds, minutes or hours as required.

The single most important thing from this lecture:

df['date'] = pd.to_datetime(df['date'])

And after that:

df['date'].dt.year
df['date'].dt.month
df['date'].dt.day
df['date'].dt.dayofweek
df['date'].dt.day_name()
df['date'].dt.quarter

---

19. Test your understanding

{@body const qs=[{q:"Why do we convert a date column using pd.to_datetime()?",o:["To remove missing values","To make Pandas recognize the column as datetime data","To convert dates into integers"],a:1,e:"Datetime conversion enables Pandas datetime operations and the .dt accessor."},{q:"Which accessor is used to extract year from a datetime column?",o:[".str.year",".dt.year",".year()"],a:1,e:"Use the .dt accessor for datetime properties."},{q:"What does dt.dayofweek return for Monday?",o:["1","7","0"],a:2,e:"Pandas uses Monday=0 and Sunday=6."},{q:"How do you identify weekends using dayofweek?",o:["Values greater than or equal to 5","Values greater than 0","Values equal to 1 or 2"],a:0,e:"Saturday is 5 and Sunday is 6."},{q:"Which property extracts the quarter?",o:["dt.month","dt.quarter","dt.week"],a:1,e:"dt.quarter returns a value from 1 to 4."},{q:"What does subtracting two datetime values produce?",o:["A string","A timedelta","A categorical variable"],a:1,e:"The difference is a timedelta, which can be converted into days or seconds."},{q:"How can you convert a timedelta into total seconds?",o:["dt.seconds","dt.total_seconds()","dt.second"],a:1,e:"total_seconds() includes the entire duration, including days."}]}
{@body const [ans,setAns]=DIL.useState({})}
{@body const [done,setDone]=DIL.useState(false)}

Lecture 34 quiz
[{Object.keys(ans).length}/{qs.length} answered]

{#each qs as q,i}

{i+1}. {q.q}
setAns(p=>({...p,[i]:Number(v)}))}>
{#each q.o as op,j}
{op}
{/each}

{#if done}

{#if ans[i]===q.a}

Correct
{:else}

Correct answer: {q.o[q.a]}
{/if}

{q.e}

{/if}

{#if i!==qs.length-1}

{/if}
{/each}
{#if done}
{qs.filter((q,i)=>ans[i]===q.a).length}/{qs.length}
{setAns({});setDone(false)}}>Retry quiz
{:else}
setDone(true)}>Submit answers
{/if}

Lecture 34 completed. The next step is to practise these datetime operations on a real dataset, especially extracting features and checking how they relate to the target variable.

[Attachment: Handling Missing Data Part 1 Complete Case Analysis.txt (53294 bytes)]

CHATGPT (Message 17)

Lecture 35 — Handling Missing Data (Part 1): Complete Case Analysis

100 Days of Machine Learning · Feature Engineering

[Complete transcript covered]

This lecture begins a new section of Feature Engineering: **Handling Missing Data**.

The central problem is simple: most machine learning algorithms cannot directly work with missing values. Before training a model, we need to decide how to handle them.

The instructor introduces two broad approaches:

1. **Removing missing data** — deleting rows or columns containing missing values.
2. **Imputation** — filling missing values with suitable replacements.

This lecture focuses on the first approach, specifically **Complete Case Analysis (CCA)**.

---

1. Why do we need to handle missing data?

Suppose you have a dataset:

Age	Salary	Experience	Purchased
25	30000	2	1
30	40000	5	0
NaN	35000	3	1
28	NaN	4	0
35	50000	8	1

Here, `NaN` represents a missing value.

Many commonly used machine learning algorithms cannot directly accept missing values in their input features.

Therefore, before training the model, we need to handle these missing values.

The instructor explains that a data scientist is responsible for preparing the dataset before passing it to a machine learning algorithm.

There are two main choices.

2. Two approaches to handling missing values

Approach 1: Removing data

Dataset containing missing values

Remove rows or columns
Discard observations containing missing data

Approach 2: Imputation

Dataset containing missing values

Fill missing values
Replace missing values using an appropriate strategy

The instructor divides imputation into two categories as well.

A. Univariate imputation

In univariate imputation, we focus on one column at a time.

For example, suppose the `Age` column contains missing values.

We can fill them using:

- Mean.
- Median.
- Random values.
- End-of-distribution values.

For categorical columns, we can use:

- Mode (most frequent value).
- A separate category such as `Missing`.

The lecture mentions that these techniques will be covered in the next video.

B. Multivariate imputation

In multivariate imputation, we use information from other columns to estimate the missing value.

The instructor introduces two techniques:

1. **KNN Imputer** — uses nearby observations to estimate missing values.
2. **Iterative Imputer** — uses a machine learning approach to estimate missing values.

These techniques are reserved for later lectures in this series.

---

3. Complete Case Analysis (CCA)

[Main topic of this lecture]

Complete Case Analysis is also called:

- Listwise deletion.
- Complete case deletion.

Definition

Complete Case Analysis is a missing-data handling technique in which we discard every observation (row) that contains at least one missing value in the selected columns.

In simple terms:

If a row has even one missing value in the columns being considered, we remove the entire row.

Let's understand this with an example.

4. How does Complete Case Analysis work?

Consider the following dataset:

Row	Age	Salary	Experience	Purchased
1	25	30000	2	1
2	30	40000	5	0
3	NaN	35000	3	1
4	28	NaN	4	0
5	35	50000	8	1

There are five observations.

- Row 1: No missing values.
- Row 2: No missing values.
- Row 3: Age is missing.
- Row 4: Salary is missing.
- Row 5: No missing values.

Using CCA, we remove rows 3 and 4.

The resulting dataset becomes:

Row	Age	Salary	Experience	Purchased
1	25	30000	2	1
2	30	40000	5	0
5	35	50000	8	1

Notice:

- Initially, there were 5 rows.
- After applying CCA, only 3 rows remain.
- The number of columns remains unchanged.

This is the basic working principle of Complete Case Analysis.

---

5. Implementing CCA using Pandas

Pandas provides a very simple function for this.

df.dropna()

This removes every row containing at least one missing value.

For example:

import pandas as pd

df = pd.read_csv('data.csv')

new_df = df.dropna()

Now `new_df` contains only complete observations.

Important parameters

The `dropna()` function has several useful parameters.

1. `axis`

df.dropna(axis=0)

- `axis=0`: Remove rows.
- `axis=1`: Remove columns.

By default, `axis=0`.

2. `how`

df.dropna(how='any')

- `how='any'`: Remove a row if at least one selected value is missing.
- `how='all'`: Remove a row only if all selected values are missing.

For standard CCA, the instructor's approach corresponds to `how='any'`.

3. `subset`

df.dropna(subset=['Age', 'Salary'])

This checks only the specified columns.

If either `Age` or `Salary` is missing, that row is removed.

This is useful when you do not want missing values in every column to determine whether a row should be deleted.

---

6. When should we apply CCA?

This is the most important conceptual part of the lecture.

The instructor explains that CCA should be used when the missing values are **Missing Completely At Random (MCAR)**.

Let's understand what that means.

Missing Completely At Random (MCAR)

MCAR means that the probability of a value being missing is unrelated to the observed data and the missing value itself.

For example, imagine a dataset with 1,000 records.

Due to a random technical issue, 50 records lose their age values.

If those 50 missing values are genuinely random, then removing those rows is less likely to introduce systematic bias.

The important point is that the missingness should not depend on a particular group, characteristic, or underlying value.

Why does MCAR matter?

Suppose the original dataset has these characteristics:

- 40% male.
- 60% female.

If missing values occur completely randomly, removing some rows should generally preserve approximately the same gender distribution.

However, suppose missing ages occur mostly among older people.

If we remove all rows with missing age values, we may remove a disproportionate number of older individuals.

The resulting dataset would no longer represent the original population properly.

This is why understanding the missingness mechanism is important before applying CCA.

The central condition

MCAR
Missingness should be completely random.

The instructor's recommendation is to consider CCA when missing values are MCAR and the amount of missing data is relatively small.

---

7. How much missing data is acceptable?

The instructor gives a practical guideline:

CCA is generally considered when missing data is around 5% or less.

This is a rule of thumb mentioned in the lecture, not a universal statistical law.

Consider a dataset with 1,000 rows.

Suppose 50 rows have missing values in a particular feature.

$$
\text{Missing Percentage}
=
\frac{\text{Missing Values}}{\text{Total Observations}}
\times 100
$$

$$
=\frac{50}{1000}\times100=5\%
$$

If we remove those 50 rows, we retain:

$$
1000-50=950
$$

Example: 5% missing data

Original rows
1,000

Rows removed
50

Rows remaining
950

95% retained

5% removed

However, if a feature has 98% missing values, CCA may be a terrible choice.

For example:

- Total rows: 1,000.
- Missing values in a feature: 980.
- Complete values: 20.

If we remove all rows with missing values in that feature, we may retain only 20 observations.

In such a situation, the instructor suggests considering removing the entire column rather than removing almost the entire dataset.

The decision depends on how important that feature is.

---

8. Advantages of Complete Case Analysis

The instructor identifies two main advantages.

8.1 Very easy to implement

CCA is simple.

You do not need to calculate means, medians, or estimates.

You can simply call:

```
df.dropna()
```

The rows containing missing values are removed.

8.2 Preserves the distribution under suitable conditions

If the missingness is genuinely MCAR, removing incomplete observations does not systematically favour particular types of observations.

Therefore, the distribution of the remaining data can remain representative of the original data.

However, you should verify this rather than assume it.

---

9. Disadvantages of Complete Case Analysis

The lecture explains three major disadvantages.

9.1 Loss of a large amount of data

This is the most obvious disadvantage.

Suppose you have 10,000 rows, but missing values are spread across many columns.

Even if each individual column has a relatively small percentage of missing values, removing every row that contains at least one missing value can result in substantial data loss.

For example:

Row	Age	Salary	Experience
1	25	30000	2
2	NaN	40000	5
3	30	NaN	3
4	28	35000	NaN
5	35	50000	8

Each of rows 2, 3, and 4 has a missing value in a different column.

CCA removes all three rows.

Only two observations remain.

This is why missing percentages should be examined across all relevant columns, not just individually.

9.2 Can distort the distribution if missingness is not random

If missingness is related to the characteristics of the observations, removing incomplete rows can change the distribution.

For example:

Suppose higher-income individuals are less likely to report their income.

Removing rows with missing income might leave you with a dataset containing a disproportionate number of lower-income individuals.

The resulting dataset would not accurately represent the original population.

9.3 The model will not learn how to handle missing values

This is an especially important point from the lecture.

Suppose you train a model after deleting every row with missing data.

During training, the model never encounters missing values.

Now imagine you deploy that model in production.

A new user submits a record with a missing age value.

Your model has not learned how to handle this situation.

The training data was cleaned, but the incoming production data may still contain missing values.

Therefore, removing missing rows from the training dataset does not automatically solve the missing-value problem in a real application.

This is one reason why imputation is often preferred.

---

10. When should you use CCA?

According to the lecture, consider CCA when:

1. Missing values are Missing Completely At Random (MCAR).
2. The proportion of missing data is relatively small — the lecture gives around 5% as a practical guideline.
3. Removing incomplete observations does not significantly change the distribution of the data.

If these conditions are not satisfied, CCA may not be appropriate.

Also, if a particular column contains an extremely high percentage of missing values, consider whether the column itself should be removed instead.

---

11. Practical implementation on a real dataset

The instructor uses a real-world job-applicant dataset.

The dataset contains information about candidates who applied for a data science job.

The columns discussed include:

Column	Description
enrollee_id	Candidate identifier
city	Candidate's city
city_development_index	Development index of the city
gender	Candidate's gender
relevant_experience	Whether the candidate has relevant experience
enrolled_university	University enrolment status
education_level	Education level
major_discipline	Candidate's major discipline
experience	Years of experience
company_size	Size of the company
company_type	Type of company
training_hours	Hours of training
target	Whether the candidate was hired

The instructor examines the missing-value percentage in each column before deciding which features are suitable for CCA.

11.1 Check missing-value percentages

The following is a clean way to calculate missing percentages:

```
missing_percent = (
df.isnull().mean() * 100
)

print(missing_percent)
```

Explanation:

- `df.isnull()` returns `True` wherever a value is missing.
- `.mean()` calculates the proportion of missing values in each column because `True` is treated as 1 and `False` as 0.
- Multiplying by 100 converts the proportions into percentages.

You can also use:

```
missing_percent.sort_values(ascending=False)
```

This makes it easier to identify columns with the most missing data.

11.2 Identify columns with less than 5% missing values

```
cols = missing_percent[
missing_percent
```

Conceptual illustration of comparing distributions; the actual plots should be generated from your dataset.

12.2 Compare categorical distributions

For categorical features, compare the proportion of each category before and after CCA.

For example, suppose `enrolled_university` contains:

- No enrollment.
- Full-time course.
- Part-time course.

We can calculate category proportions:

```
before = (
df['enrolled_university']
.value_counts(normalize=True)
)

after = (
new_df['enrolled_university']
```

.value_counts(normalize=True)
)

print(before)
print(after)

The parameter:

normalize=True

returns proportions instead of raw counts.

Example:

Category	Before CCA	After CCA
No enrollment	72%	72.7%
Full-time course	19%	19.2%
Part-time course	6%	6.0%
Other	3%	2.1%

These are illustrative values, not the exact dataset output.

The instructor's point is that category proportions should remain approximately similar after CCA.

If the proportions change substantially, it may indicate that removing incomplete rows has affected the distribution.

---

13. Complete CCA workflow

Let's combine the important operations.

import pandas as pd
import matplotlib.pyplot as plt

Load dataset
df = pd.read_csv('data.csv')

1. Calculate missing percentages
missing_percent = df.isnull().mean() * 100

2. Select columns with less than 5% missing values
cols = missing_percent[
missing_percent
Calculate missing-value percentages.
Check whether the missingness is plausibly MCAR.
Check how many rows will be removed.
Apply CCA only when the loss of information is acceptable.
Compare numerical distributions and categorical proportions before and after deletion.

Plan how missing values will be handled in production data.

---

17. Test your understanding

{@body const qs=[{q:"What is Complete Case Analysis?",o:["Filling missing values with mean","Removing rows containing missing values","Removing all numerical columns"],a:1,e:"CCA, also called listwise deletion, removes incomplete observations."},{q:"What is another name for CCA?",o:["Random imputation","Listwise deletion","Mean encoding"],a:1,e:"Complete Case Analysis is also called listwise deletion."},{q:"According to the lecture, which missingness mechanism is suitable for CCA?",o:["MCAR","Every missingness mechanism","Only missing values in target"],a:0,e:"The lecture recommends CCA when data is Missing Completely At Random."},{q:"What is the lecture's practical missing-data guideline for CCA?",o:["Up to 50%","Around 5% or less","Exactly 25%"],a:1,e:"The lecture gives approximately 5% as a rule of thumb, not a universal statistical law."},{q:"Which Pandas function removes rows with missing values?",o:["fillna()","dropna()","isnull()"],a:1,e:"dropna() removes rows containing missing values by default."},{q:"How should numerical distributions be compared before and after CCA?",o:["Using histograms","Using only row counts","Using string extraction"],a:0,e:"Histograms help compare the shape of numerical distributions."},{q:"How should categorical distributions be compared?",o:["Using category proportions","Using standard deviation","Using numerical scaling"],a:0,e:"Compare category proportions using value_counts(normalize=True)."}]}
{@body const [ans,setAns]=DIL.useState({})}
{@body const [done,setDone]=DIL.useState(false)}

Lecture 35 quiz
[{Object.keys(ans).length}/{qs.length} answered]

{#each qs as q,i}

{i+1}. {q.q}
setAns(p=>({...p,[i]:Number(v)}))}>
{#each q.o as op,j}
{op}
{/each}

{#if done}

{#if ans[i]===q.a}

Correct
{:else}

Correct answer: {q.o[q.a]}
{/if}

{q.e}

{/if}

{#if i!==qs.length-1}

{/if}
{/each}
{#if done}
{qs.filter((q,i)=>ans[i]===q.a).length}/{qs.length}
{setAns({});setDone(false)}}>Retry quiz
{:else}
setDone(true)}>Submit answers
{/if}

Lecture 35 completed.

The next lecture in the series moves to **Univariate Imputation using SimpleImputer**, where instead of deleting incomplete rows, we learn how to fill missing values using statistical techniques.

USER (Message 18)

Create a pdf file thst i can read of the chats without excluding single line

Create in the same format like chat response image and everything

CHATGPT (Message 19)

I can create a **PDF that looks like the ChatGPT conversation**, rather than a plain document.

I understand you want:
- Every message and every line preserved — no summarising or shortening.
- The same headings, bold text, tables, code blocks, lists and spacing.
- Images included in their original positions wherever available.
- Separate visual styling for your messages and my responses.
- A readable layout suitable for offline study.

I believe you mean the ML lecture-note conversations for **Lectures 31–35** (Power Transformation through Missing Data Part 1).

One thing I need: **Do you want only Lectures 31–35, or the entire conversation history?**

For an exact, line-by-line copy, I'll need the original chat content or a ChatGPT data export; I currently have summaries of some earlier responses, not every original line and image.

USER (Message 20)

Entire convo history

CHATGPT (Message 21)

Understood — you want **your entire ChatGPT conversation history**, not just the ML lectures, turned into a PDF.

To preserve every line accurately, including images and the original message order, the best option is to use your ChatGPT data export. I can't directly retrieve every conversation in your account from this chat.

CHATGPT (Message 22)

Understood — you want **your entire ChatGPT conversation history**, not just the ML lectures, turned into a PDF.

To preserve every line accurately, including images and the original message order, the best option is to use your ChatGPT data export. I can't directly retrieve every conversation in your account from this chat.

How to send me your conversation history

On your Android app:

1. Open **ChatGPT Settings**.
2. Go to **Data Controls**.
3. Select **Export Data**.
4. Confirm the export request.
5. Check your registered email for the download link.
6. Download the ZIP file and upload it here.

Please upload the original ZIP file, rather than copying and pasting conversations. It gives me the best chance of preserving message formatting, timestamps, and images.

What I'll create

- ChatGPT-style conversation layout.
- Your messages and my replies visually separated.
- Original text, code, tables, headings and images.
- Conversations in chronological order.
- No intentional summarisation or removal of messages.
- Page numbers and a table of contents.

If the history is too large for one PDF, I'll split it into numbered volumes so it remains readable.

I'll preserve the original content rather than reconstructing missing messages from memory. Upload the ZIP whenever you're ready.

If you want, I can:

- Explain limitations of Complete Case Analysis
- Describe how to verify MCAR assumption
- Outline steps to implement CCA in a data project

CHATGPT (Message 23)

How to send me your conversation history

On your Android app:

1. Open **ChatGPT Settings**.
2. Go to **Data Controls**.
3. Select **Export Data**.
4. Confirm the export request.
5. Check your registered email for the download link.
6. Download the ZIP file and upload it here.

Please upload the original ZIP file, rather than copying and pasting conversations. It gives me the best chance of preserving message formatting, timestamps, and images.

What I'll create

- ChatGPT-style conversation layout.
- Your messages and my replies visually separated.
- Original text, code, tables, headings and images.
- Conversations in chronological order.
- No intentional summarisation or removal of messages.
- Page numbers and a table of contents.

If the history is too large for one PDF, I'll split it into numbered volumes so it remains readable.

Iâ■■ll preserve the original content rather than reconstructing missing messages from memory. Upload the ZIP whenever you're ready.
